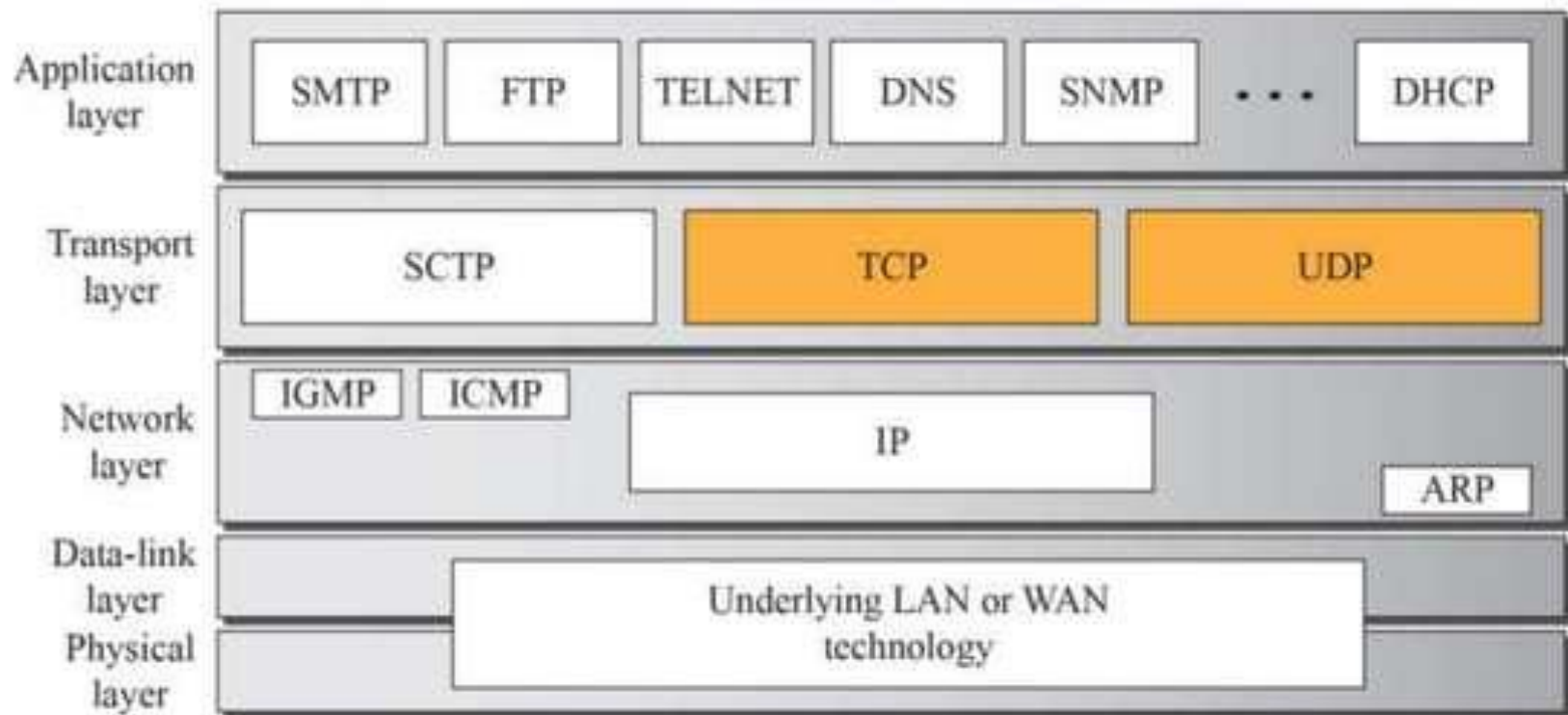


Unit 4

Transport Layer

Fig 24.1: *Position of transport-layer protocols in the TCP/IP protocol suite*





24.24.2 Port Numbers

Transport-layer protocol usually has several responsibilities.

One is to create a process-to-process communication; these protocols use port numbers to accomplish this.

Port numbers provide end-to-end addresses at the transport layer and allow multiplexing and demultiplexing at this layer, just as IP addresses do at the network layer.

Table 24.1 gives some common port numbers for all three protocols discussed.

Table 24.1: Some well-known ports used with UDP and TCP

<i>Port</i>	<i>Protocol</i>	<i>UDP</i>	<i>TCP</i>	<i>Description</i>
7	Echo	√		Echoes back a received datagram
9	Discard	√		Discards any datagram that is received
11	Users	√	√	Active users
13	Daytime	√	√	Returns the date and the time
17	Quote	√	√	Returns a quote of the day
19	Chargen	√	√	Returns a string of characters
20, 21	FTP		√	File Transfer Protocol
23	TELNET		√	Terminal Network
25	SMTP		√	Simple Mail Transfer Protocol
53	DNS	√	√	Domain Name Service
67	DHCP	√	√	Dynamic Host Configuration Protocol
69	TFTP	√		Trivial File Transfer Protocol
80	HTTP		√	Hypertext Transfer Protocol
111	RPC	√	√	Remote Procedure Call
123	NTP	√	√	Network Time Protocol
161, 162	SNMP		√	Simple Network Management Protocol

24-2 UDP

The User Datagram Protocol (UDP) is a connectionless, unreliable transport protocol. If UDP is so powerless, why would a process want to use it? With the disadvantages come some advantages. UDP is a very simple protocol using a minimum of overhead.

If a process wants to send a small message and does not care much about reliability, it can use UDP. Sending a small message using UDP takes much less interaction between the sender and receiver than using TCP.



24.2.1 User Datagram

UDP packets, called user datagrams, have a fixed-size header of 8 bytes made of four fields, each of 2 bytes (16 bits). Figure 24.2 shows the format of a user datagram. The first two fields define the source and destination port numbers. The third field defines the total length of the user datagram, header plus data. The 16 bits can define a total length of 0 to 65,535 bytes. However, the total length needs to be less because a UDP user datagram is stored in an IP datagram with the total length of 65,535 bytes.

UDP

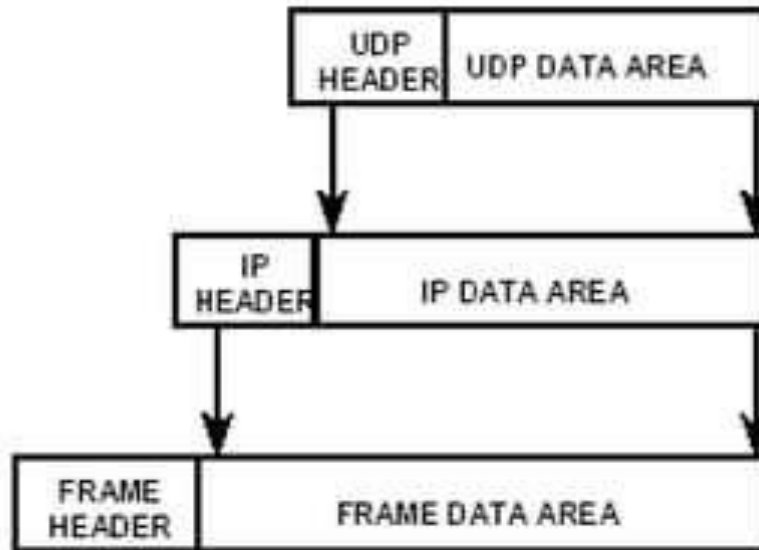
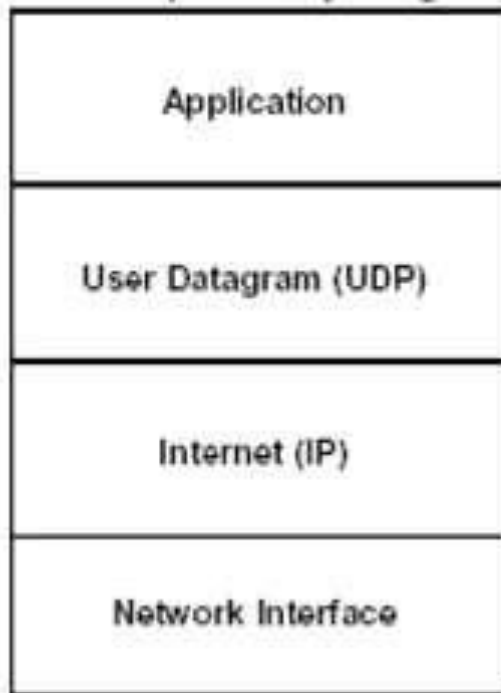
- The **User Datagram Protocol (UDP)** is a transport layer protocol defined for use with the IP network layer protocol. It is defined by RFC 768 written by John Postel. It provides a best-effort datagram service to an End System (IP host).
- The service provided by UDP is an unreliable service that provides no guarantees for delivery and no protection from duplication.
- The simplicity of UDP reduces the overhead from using the protocol and the services may be adequate.
- UDP provides minimal, unreliable, best effort, message passing transport to applications and upper layer protocols.

Encapsulation and Layering



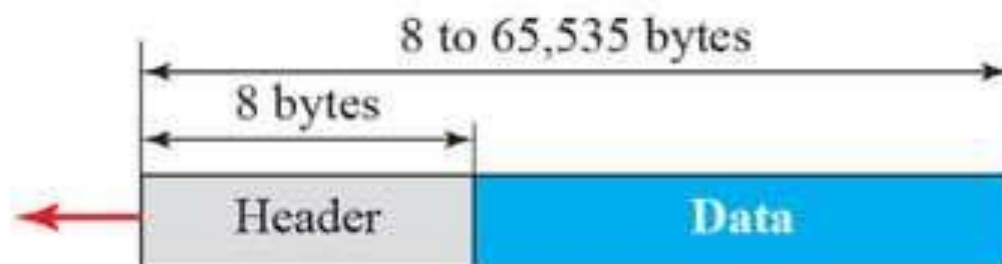
- Protocol layering
- UDP encapsulation

Conceptual Layering

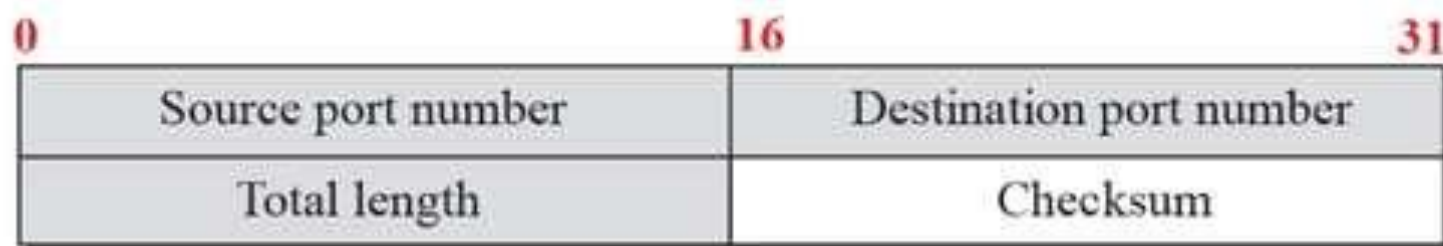


- UDP message is encapsulated into an IP datagram.
- IP datagram in turn is encapsulated into a physical frame for actually delivery.

Figure 24.2: User datagram packet format



a. UDP user datagram

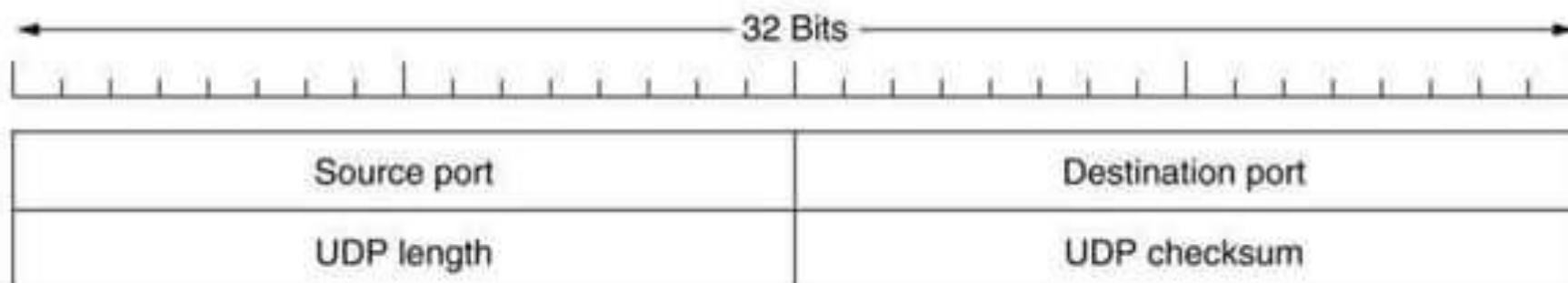


b. Header format

Introduction to UDP

The Internet protocol suite supports a connectionless transport protocol, UDP (User Datagram Protocol).

The UDP header.



UDP provides a way for applications to send encapsulated IP datagrams and send them without having to establish a connection.

UDP transmits segments consisting of an 8-byte header followed by the payload. The two ports serve to identify the end points within the source and destination machines. When a UDP packet arrives, its payload is handed to the process attached to the destination port.

The source port is primarily needed when a reply must be sent back to the source. The UDP length field includes the 8-byte header and the data.

Flow Control

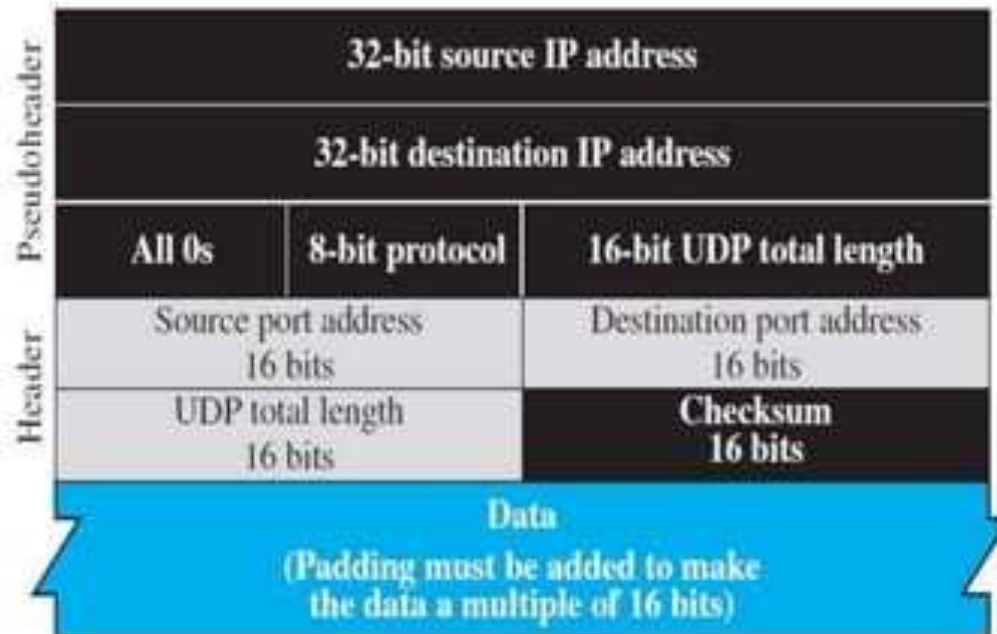
UDP is a very simple protocol. There is no *flow control*, and hence no window mechanism. The receiver may overflow with incoming messages. UDP should provide for this service, if needed.

Error Control

There is no *error control* mechanism in UDP except for the checksum. This means that the sender does not know if a message has been lost or duplicated. When the receiver detects an error through the checksum, the user datagram is silently discarded.

Checksum

UDP checksum calculation includes three sections: a pseudo header, the UDP header, and the data coming from the application layer. The *pseudo header* is the part of the header of the IP packet in which the user datagram is to be encapsulated with some fields filled with 0s.



If the checksum does not include the pseudoheader, a user datagram may arrive safe and sound. However, if the IP header is corrupted, it may be delivered to the wrong host.

The protocol field is added to ensure that the packet belongs to UDP, and not to TCP.

Congestion Control

Since UDP is a connectionless protocol, it does not provide congestion control.

UDP

assumes that the packets sent are small and sporadic and cannot create congestion in the network.

Encapsulation and Decapsulation

To send a message from one process to another, the UDP protocol encapsulates and decapsulates messages.

Queuing

In UDP, queues are associated with ports. At the client site, when a process starts, it requests a port number from the operating system.

Some implementations create both an incoming and an outgoing queue associated with each process. Other implementations create only an incoming queue associated with each process.

Multiplexing and Demultiplexing

In a host running a TCP/IP protocol suite, there is only one UDP but possibly several processes that may want to use the services of UDP. To handle this



24.2.3 UDP Applications

Although UDP meets almost none of the criteria we mentioned earlier for a reliable transport-layer protocol, UDP is preferable for some applications.

The reason is that some services may have some side effects that are either unacceptable or not preferable. An application designer sometimes needs to compromise to get the optimum.

For example, in our daily life, we all know that a one-day delivery of a package by a carrier is more expensive than a three-day delivery. Although high speed and low cost are both desirable features in delivery of a parcel, they are in conflict with each other.

UDP Features

Some of the features of UDP and their advantages and disadvantages.

Connectionless Service

As we mentioned previously, UDP is a connectionless protocol. Each UDP packet is independent from other packets sent by the same application program.

Lack of Error Control

UDP does not provide error control; it provides an unreliable service. Most applications expect reliable service from a transport-layer protocol.

Lack of Congestion Control

UDP does not provide congestion control. However, UDP does not create additional traffic in an error-prone network. TCP may resend a packet several times and thus contribute to the creation of congestion or worsen a congested situation.

Typical Applications

The following shows some typical applications that can benefit more from the services of UDP than from those of TCP.

UDP is suitable for a process that requires simple request-response communication with little concern for flow and error control. It is not usually used for a process such as FTP that needs to send bulk data.

UDP is suitable for a process with internal flow- and error-control mechanisms. For example, the Trivial File Transfer Protocol (TFTP) process includes flow and error control.

UDP is a suitable transport protocol for multicasting. Multicasting capability is embedded in the UDP software but not in the TCP software.

UDP is used for management processes such as SNMP.

Used for some route updating protocols such as Routing Information Protocol (RIP).

UDP is normally used for interactive real-time applications that cannot tolerate uneven delay between sections of a received message.

TRANSMISSION CONTROL PROTOCOL

Transmission Control Protocol (TCP) is a connection-oriented, reliable protocol. TCP explicitly defines connection establishment, data transfer, and connection teardown phases to provide a connection-oriented service.

TCP uses a combination of GBN and SR protocols to provide reliability. To achieve this goal, TCP uses checksum (for error detection), retransmission of lost or corrupted packets, cumulative and selective acknowledgments, and timers.

TCP Services

The services offered by TCP to the processes at the application layer.

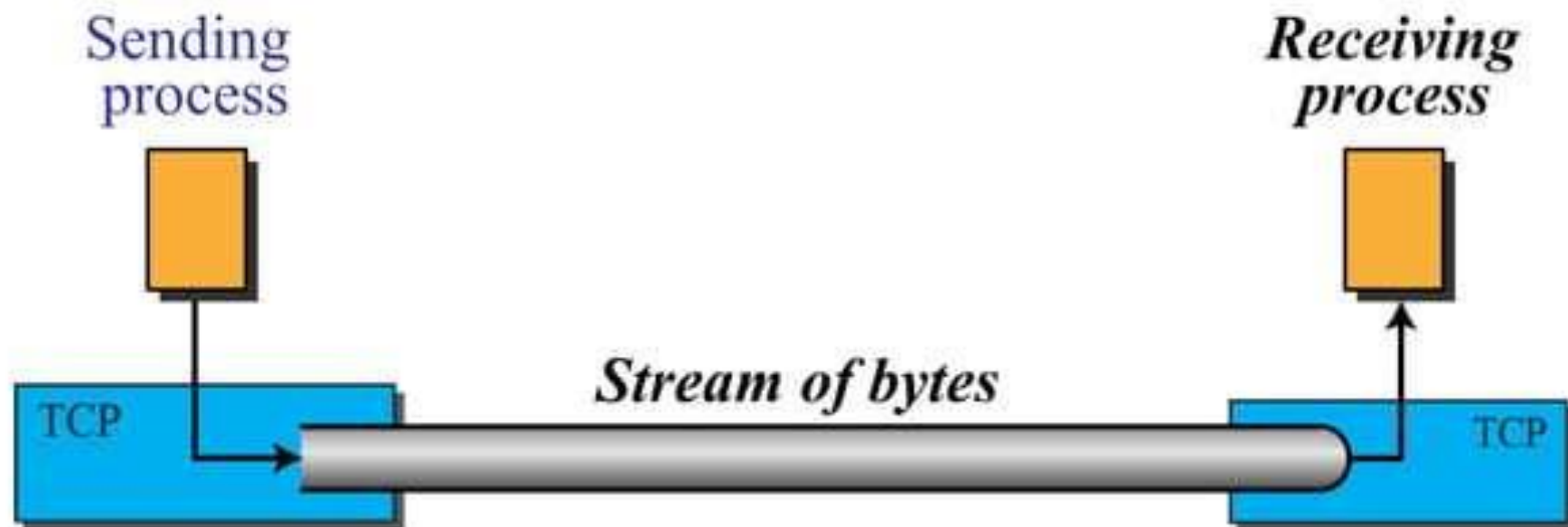
Process-to-Process Communication

As with UDP, TCP provides process-to-process communication using port numbers.

Stream Delivery Service

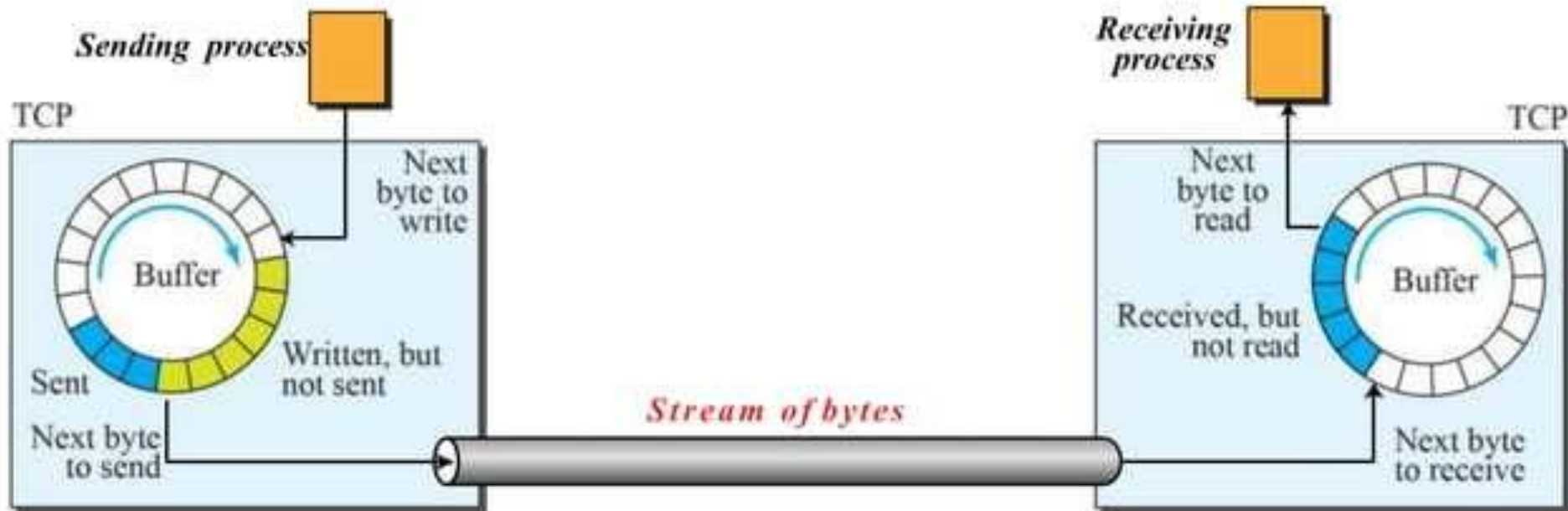
TCP, unlike UDP, is a stream-oriented protocol. In UDP, a process sends messages with predefined boundaries to UDP for delivery. UDP adds its own header to each of these messages and delivers it to IP for transmission. Each message from the process is called a *user datagram*, and becomes, eventually, one IP datagram. Neither IP nor UDP recognizes any relationship between the datagrams.

Stream delivery



TCP, on the other hand, allows the sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as a stream of bytes. TCP creates an environment in which the two processes seem to be connected by an imaginary “tube” that carries their bytes across the Internet. This imaginary environment is depicted in Figure. The sending process produces (writes to) the stream and the receiving process consumes (reads from) it.

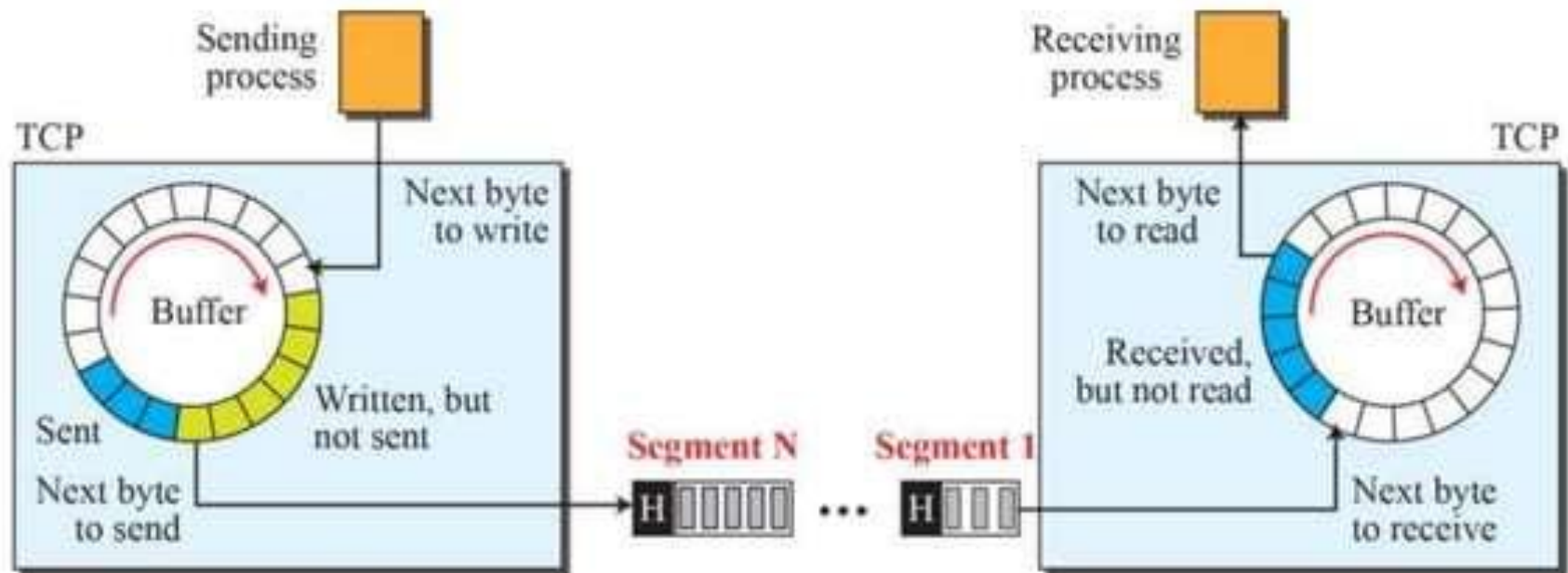
Sending and receiving buffers



At the sender, the buffer has three types of chambers. The white section contains empty chambers that can be filled by the sending process (producer). The colored area holds bytes that have been sent but not yet acknowledged. The TCP sender keeps these bytes in the buffer until it receives an acknowledgment. The shaded area contains bytes to be sent by the sending TCP.

The operation of the buffer at the receiver is simpler. The circular buffer is divided into two areas (shown as white and colored). The white area contains empty chambers to be filled by bytes received from the network. The colored sections contain received bytes that can be read by the receiving process.

TCP segments



At the transport layer, TCP groups a number of bytes together into a packet called a *segment*. TCP adds a header to each segment (for control purposes) and delivers the segment to the network layer for transmission.

The segments are encapsulated in an IP datagram and transmitted. Note that segments are not necessarily all the same size. In the figure, one segment carries 3 bytes and the other is carrying 5 bytes.

Full-Duplex Communication

TCP offers *full-duplex service*, where data can flow in both directions at the same time. Each TCP endpoint then has its own sending and receiving buffer, and segments move in both directions.

Multiplexing and Demultiplexing

Like UDP, TCP performs multiplexing at the sender and demultiplexing at the receiver. However, since TCP is a connection-oriented protocol, a connection needs to be established for each pair of processes.

Connection-Oriented Service

TCP, unlike UDP, is a connection-oriented protocol. When a process at site A wants to send to and receive data from another process at site B, the following three phases occur:

1. The two TCP's establish a logical connection between them.
2. Data are exchanged in both directions.
3. The connection is terminated.

Reliable Service

TCP is a reliable transport protocol. It uses an acknowledgment mechanism to check the safe and sound arrival of data.

TCP Features

To provide the services mentioned in the previous section, TCP has several features

Numbering System

Although the TCP software keeps track of the segments being transmitted or received, there is no field for a segment number value in the segment header. Instead, there are two fields, called the *sequence number* and the *acknowledgment number*.

Byte Number

TCP numbers all data bytes (octets) that are transmitted in a connection. Numbering is independent in each direction. The numbering does not necessarily start from 0. Instead, TCP chooses an arbitrary number between 0 and $2^{32} - 1$ for the number of the first byte. For example, if the number happens to be 1057 and the total data to be sent is 6000 bytes, the bytes are numbered from 1057 to 7056.

Sequence Number

After the bytes have been numbered, TCP assigns a sequence number to each segment that is being sent. The sequence number, in each direction, is defined as follows:

1. The sequence number of the first segment is the ISN (initial sequence number), which is a random number.
2. The sequence number of any other segment is the number of the previous segment plus the number of bytes (real or imaginary) carried by the previous segment.

Acknowledgment Number

The value of the acknowledgment field in a segment defines the number of the next byte

24.22 party expects to receive. The acknowledgment number is cumulative.

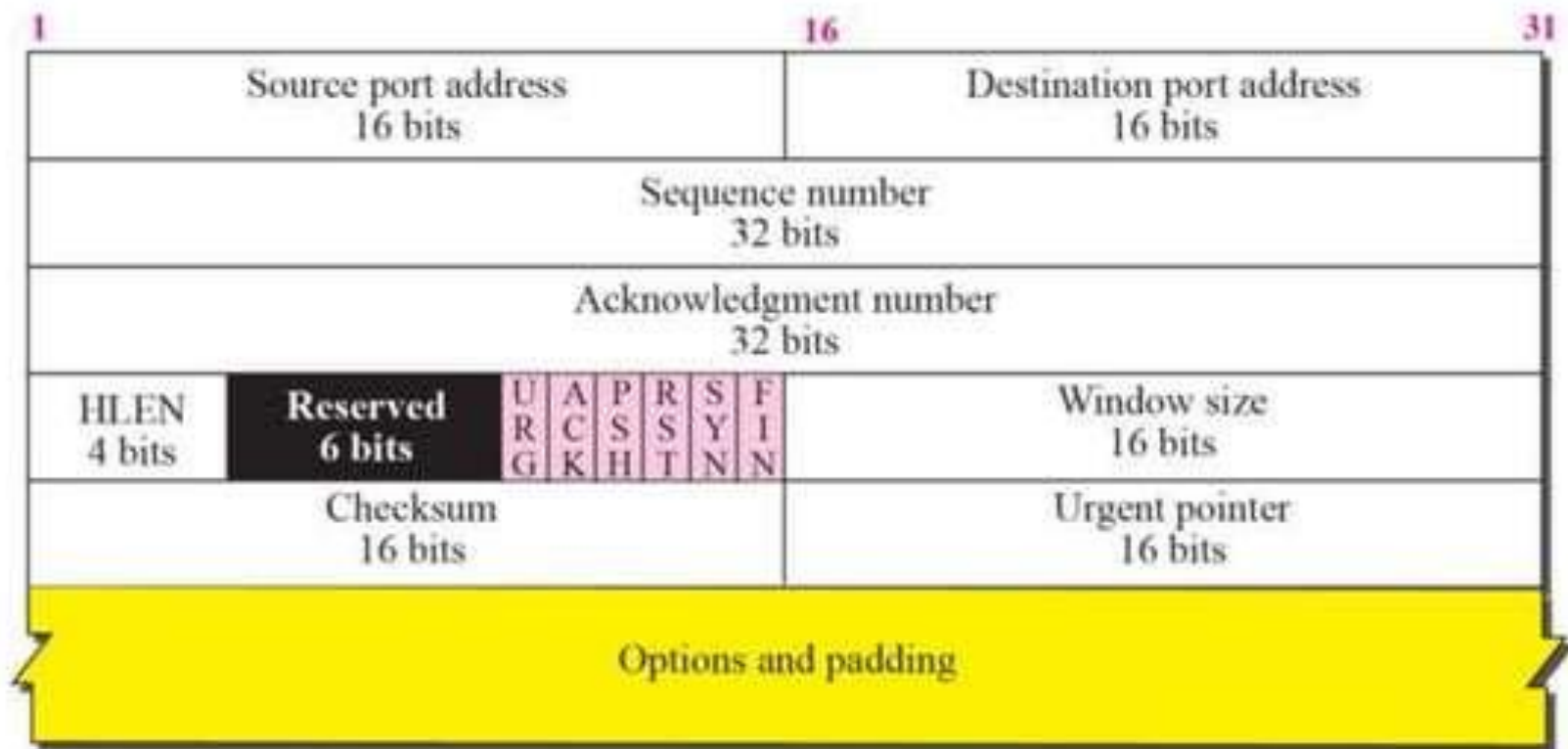
The TCP Protocol

- Every byte on a TCP connection has its own 32-bit sequence number.
- The sending and receiving TCP entities exchange data in the form of segments. A **TCP segment** consists of a fixed 20-byte header (plus an optional part) followed by zero or more data bytes. The TCP software decides how big segments should be.
- Two limits restrict the segment size. First, each segment, including the TCP header, must fit in the 65,515-byte IP payload. Second, each network has a maximum transfer unit, or MTU, and each segment must fit in the MTU

TCP segment format



a. Segment



b. Header

- The Source port and Destination port fields identify the local end points of the connection. A port plus its host's IP address forms a 48-bit unique end point. The source and destination end points together identify the connection.
- This connection identifier is called a **5 tuple** because it consists of five pieces of information: the protocol (TCP), source, IP and source port, and destination IP and destination port.
- The Sequence number and Acknowledgement number fields perform their usual functions. The latter specifies the next byte expected, not the last byte correctly received. Both are 32 bits long because every byte of data is numbered in a TCP stream.
- The TCP header length tells how many 32-bit words are contained in the TCP header.
- Next comes a 6-bit field that is not used.

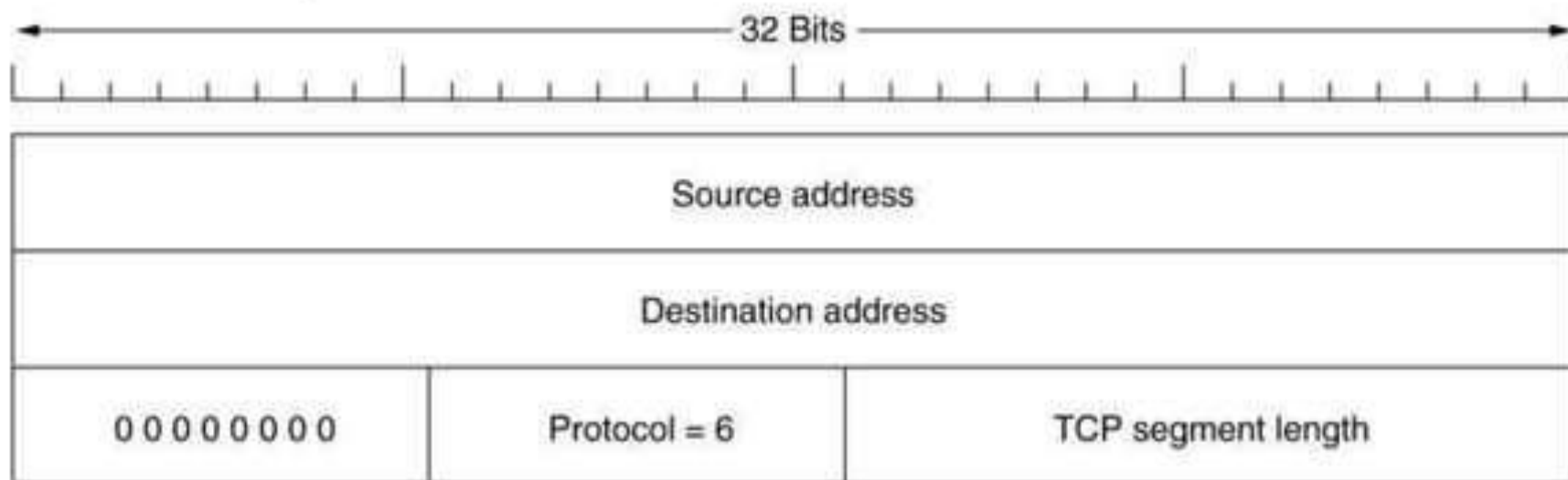
- *CWR and ECE are used to signal congestion when ECN (Explicit Congestion Notification) is used. ECE is set to signal an ECN-Echo to a TCP sender to tell it to slow down when the TCP receiver gets a congestion indication from the network. CWR is set to signal Congestion Window Reduced from the TCP sender to the TCP receiver so that it knows the sender has slowed down and can stop sending the ECN-Echo.*
- *URG is set to 1 if the Urgent pointer is in use. The Urgent pointer is used to indicate a byte offset from the current sequence number at which urgent data are to be found.*
- *The ACK bit is set to 1 to indicate that the Acknowledgement number is valid.*

- The *PSH* bit indicates *PUSHed* data. The receiver is hereby kindly requested to deliver the data to the application upon arrival and not buffer it until a full buffer has been received.
- The *RST* bit is used to abruptly reset a connection that has become confused due to a host crash or some other reason.
- The *SYN* bit is used to establish connections. The connection request has $SYN = 1$ and $ACK = 0$. The connection reply does bear an acknowledgement, however, so it has $SYN = 1$ and $ACK = 1$.
- The *FIN* bit is used to release a connection. It specifies that the sender has no more data to transmit.

- Flow control in TCP is handled using a variable-sized sliding window. The *Window size field* tells how many bytes may be sent starting at the byte acknowledged.
- A *Checksum* is also provided for extra reliability. It checksums the header, the data, and a conceptual pseudo header in exactly the same way as UDP.
- The *Options field* provides a way to add extra facilities not covered by the regular header. Many options have been defined and several are commonly used.
 - A widely used option is the one that allows each host to specify the **MSS (Maximum Segment Size)** it is willing to accept.
 - The **timestamp option** carries a timestamp sent by the sender and echoed by the receiver.
 - The **SACK (Selective ACKnowledgement)** option lets a receiver tell a sender the ranges of sequence numbers that it has received

The TCP Segment Header

The pseudoheader included in the TCP





24.3.4 A TCP Connection

TCP is connection-oriented. All of the segments belonging to a message are then sent over this logical path.

Using a single logical pathway for the entire message facilitates the acknowledgment process as well as retransmission of damaged or lost frames.

TCP, which uses the services of IP, a connectionless protocol, can be connection-oriented. The point is that a TCP connection is logical, not physical.

TCP uses the services of IP to deliver individual segments to the receiver, but it controls the connection itself.

Connection Establishment

TCP transmits data in full-duplex mode. When two TCPs in two machines are connected, they are able to send segments to each other simultaneously.

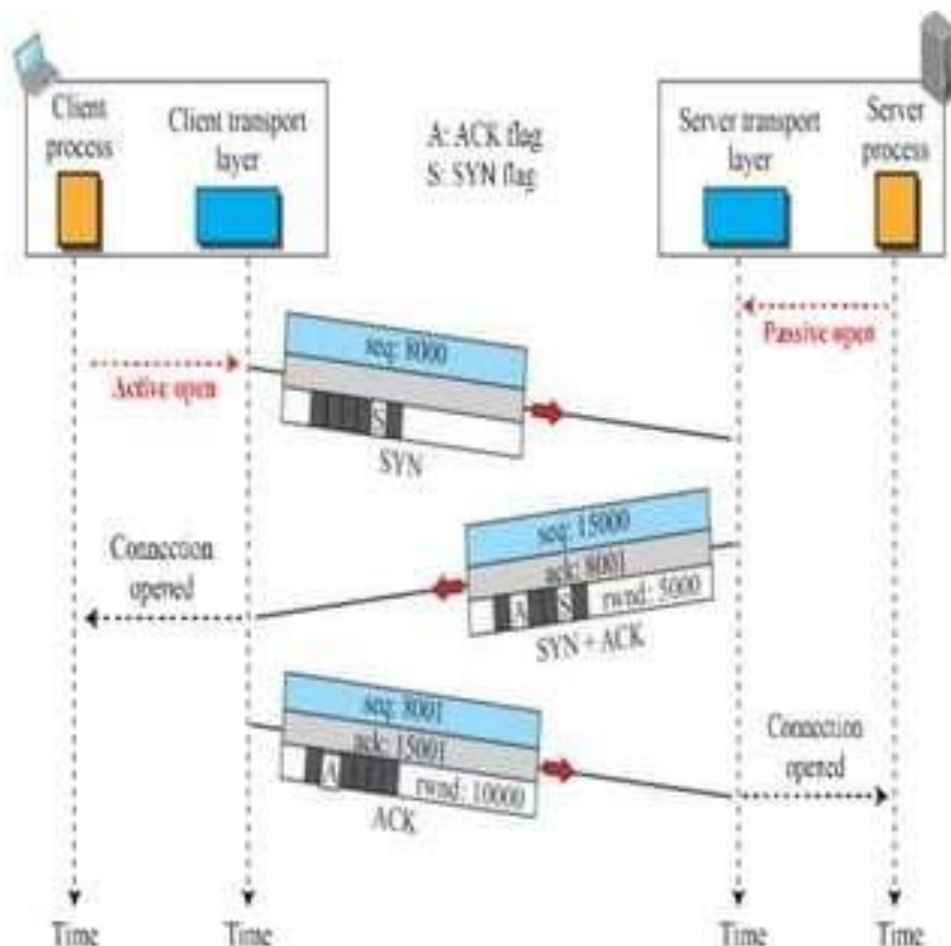
Three-Way Handshaking

The connection establishment in TCP is called *three-way handshaking*. In our example, an application program, called the *client*, wants to make a connection with another application program, called the *server*, using TCP as the transport-layer protocol.

The process starts with the server. The server program tells its TCP that it is ready to accept a connection. This request is called a *passive open*. Although the server TCP is ready to accept a connection from any machine in the world, it cannot make the connection itself.

The client program issues a request for an *active open*. A client that wishes to connect to an open server tells its TCP to connect to a particular server.

Figure 24.10: Connection establishment using three-way handshaking



•**Step 1 (SYN)** : In the first step, client wants to establish a connection with server, so it sends a segment with SYN(Synchronize Sequence Number) which informs server that client is likely to start communication and with what sequence number it starts segments with

•**Step 2 (SYN + ACK)**: Server responds to the client request with SYN-ACK signal bits set. Acknowledgement(ACK) signifies the response of segment it received and SYN signifies with what sequence number it is likely to start the segments with

•**Step 3 (ACK)** : In the final part client acknowledges the response of server and they both establish a reliable connection with which they will start the actual data transfer

SYN Attack

- Attacker A initiates a SYN flooding by generating many requests with SPOOFED source address.
- Thus forces 'D' to allocate resources.
- With many such requests destination host can run out of resources.
-DOS.....DOS.....DOS.....

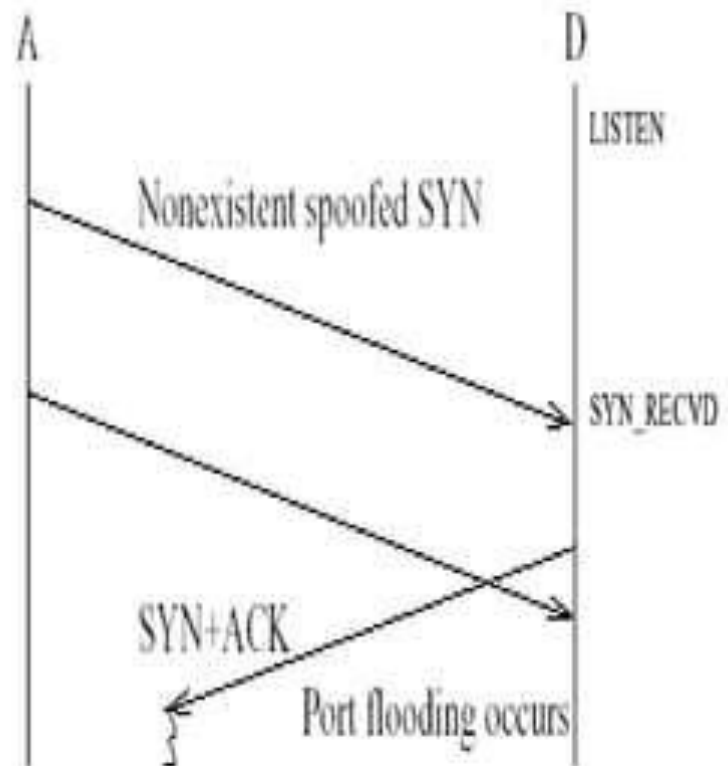
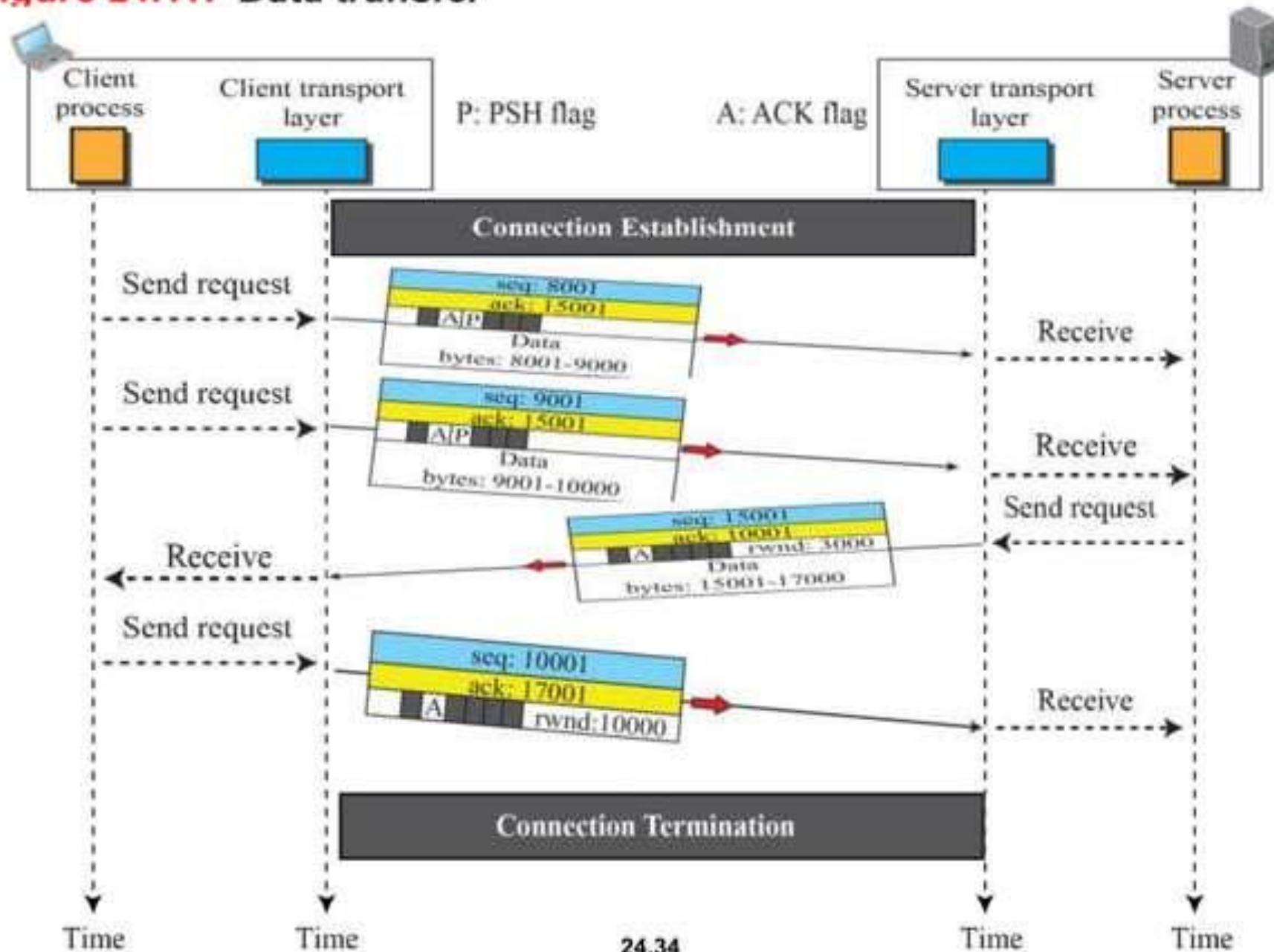


Figure 24.11: Data transfer



Pushing Data

The sending TCP uses a buffer to store the stream of data coming from the sending application program.

However, there are occasions in which the application program has no need for this flexibility. The application program on one site wants to send a chunk of data to the application program at the other site and receive an immediate response.

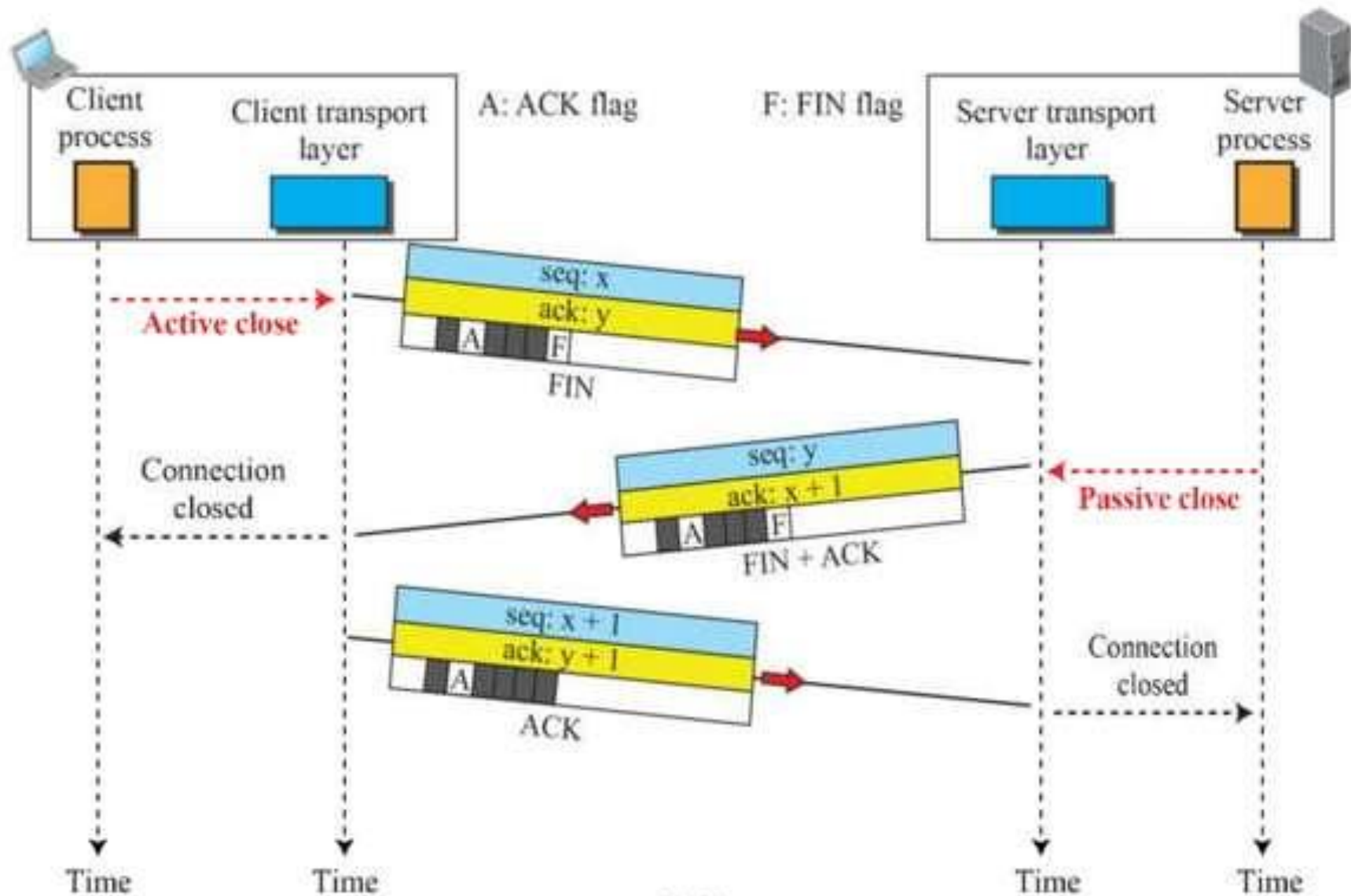
Delayed transmission and delayed delivery of data may not be acceptable by the application program.

TCP can handle such a situation. The application program at the sender can request a *push* operation. This means that the sending TCP must not wait for the window to be filled. It must create a segment and send it immediately

Urgent Data

TCP is a stream-oriented protocol. This means that the data is presented from the application program to TCP as a stream of bytes. Each byte of data has a position in the stream. However, there are occasions in which an application program needs to send *urgent* bytes, some bytes that need to be treated in a special way by the application at the other end. The solution is to send a segment with the URG bit set.

Figure 24.12: Connection termination using three-way handshaking

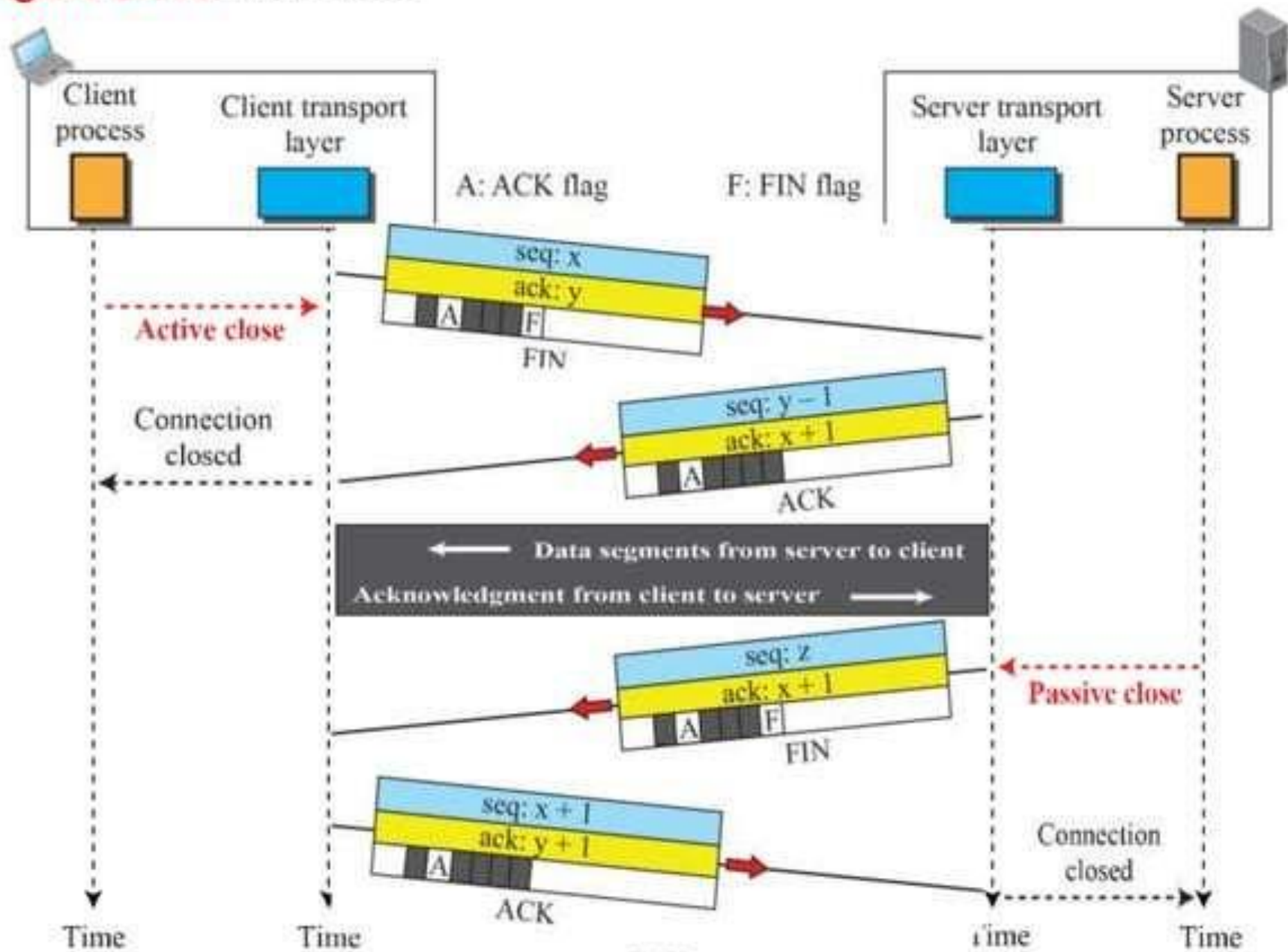


Three-Way Handshaking for connection termination.

Most implementations today allow *three-way handshaking* for connection termination,

1. The client TCP, after receiving a close command from the client process, sends the first segment, a FIN segment in which the FIN flag is set. If it is only a control segment, it consumes only one sequence number because it needs to be acknowledged.
2. The server TCP, after receiving the FIN segment, informs its process of the situation and sends the second segment, a FIN + ACK segment, to confirm the receipt of the FIN segment from the client and at the same time to announce the closing of the connection in the other direction. If it does not carry data, it consumes only one sequence number because it needs to be acknowledged.
3. The client TCP sends the last segment, an ACK segment, to confirm the receipt of the FIN segment from the TCP server. This segment contains the acknowledgment number, which is one plus the sequence number received in the FIN segment from the server. This segment cannot carry data and consumes no sequence numbers.

Figure 24.13: Half-close



Half-Close

In TCP, one end can stop sending data while still receiving data. This is called a *half close*. Either the server or the client can issue a half-close request. It can occur when the server needs all the data before processing can begin.

This means the client, after sending all data, can close the connection in the client-to-server direction.

However, the server-to-client direction must remain open to return the data.

The data transfer from the client to the server stops. The client half-closes the connection by sending a FIN segment. The server accepts the half-close by sending the ACK segment. The server, however, can still send data.

When the server has sent all of the processed data, it sends a FIN segment, which is acknowledged by an ACK from the client.

After half-closing the connection, data can travel from the server to the client and acknowledgments can travel from the client to the server. The client cannot send any more data to the server.

Connection Reset

TCP at one end may deny a connection request, may abort an existing connection, or may terminate an idle connection. All of these are done with the RST (reset) flag.



24.3.6 Windows in TCP

TCP uses two windows (send window and receive window) for each direction of data transfer, which means four windows for a bidirectional communication.

To make the discussion simple, we make an unrealistic assumption that communication is only unidirectional. The bidirectional communication can be inferred using two unidirectional communications with piggybacking.

Figure 24.17: Send window in TCP

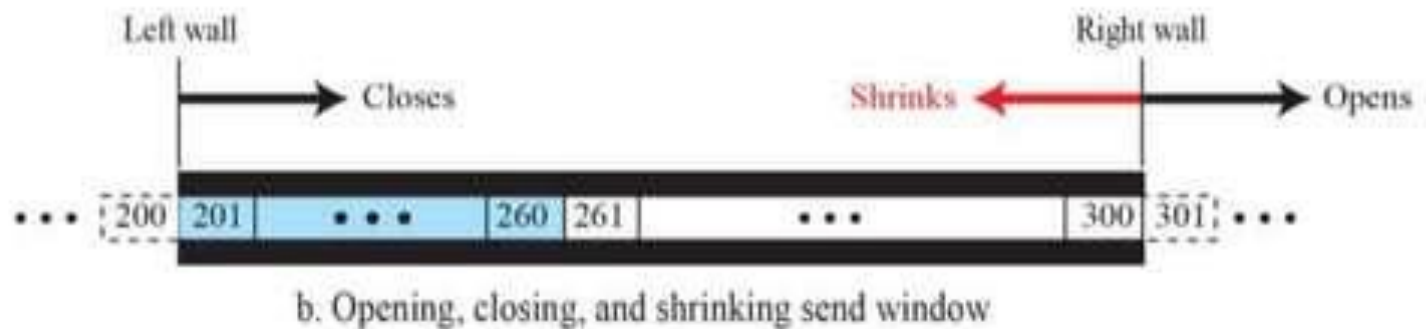
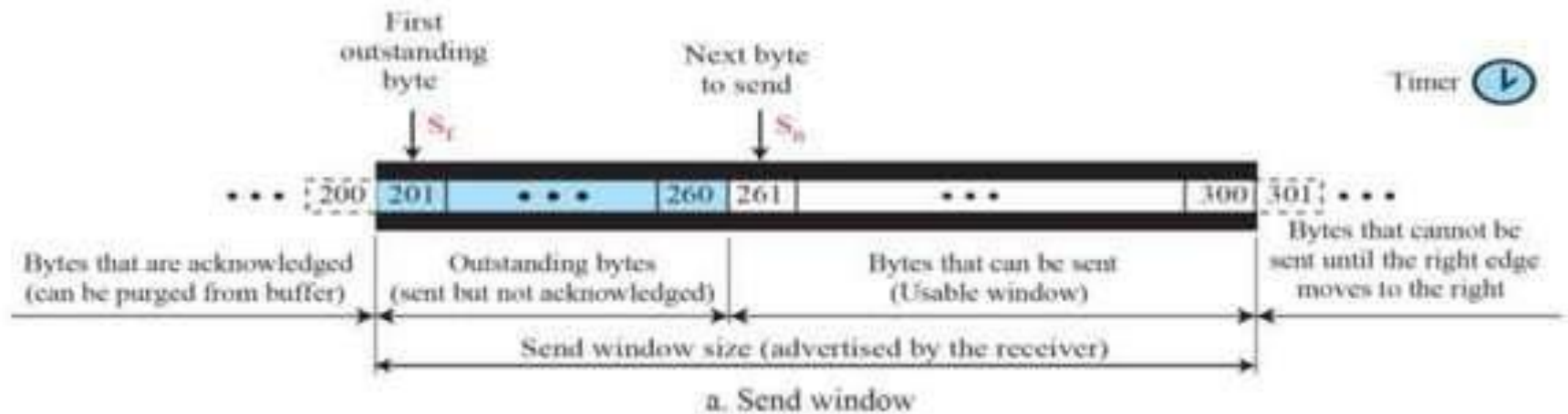
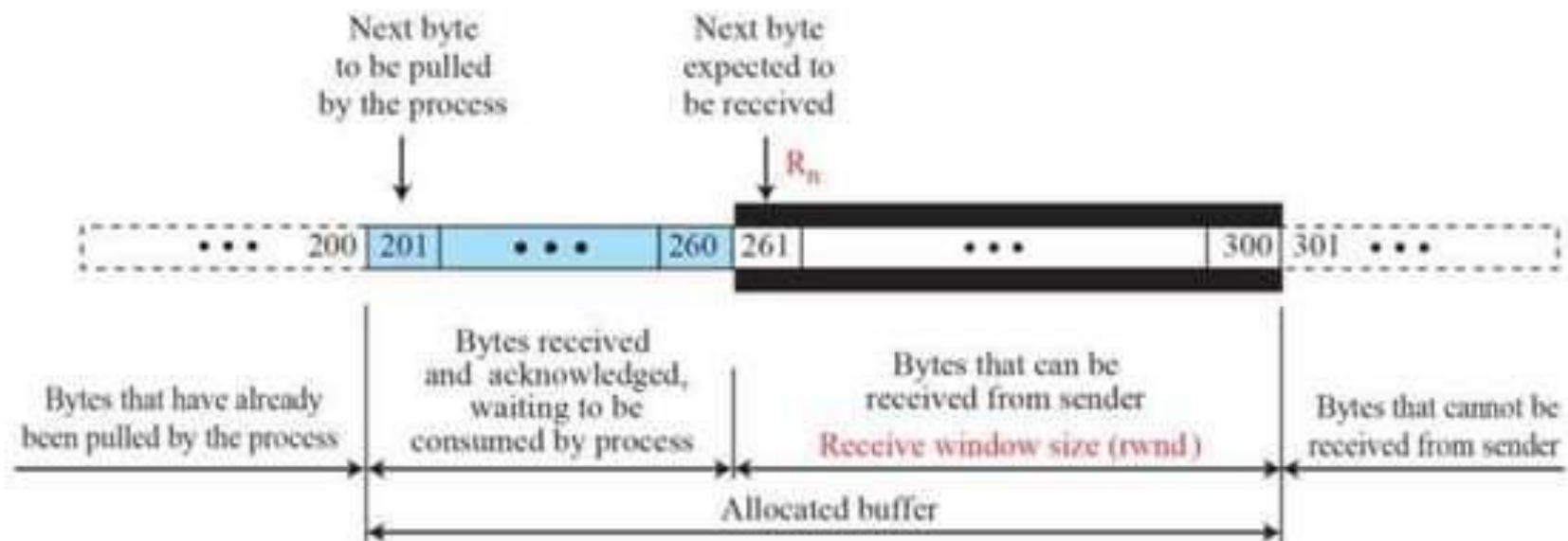
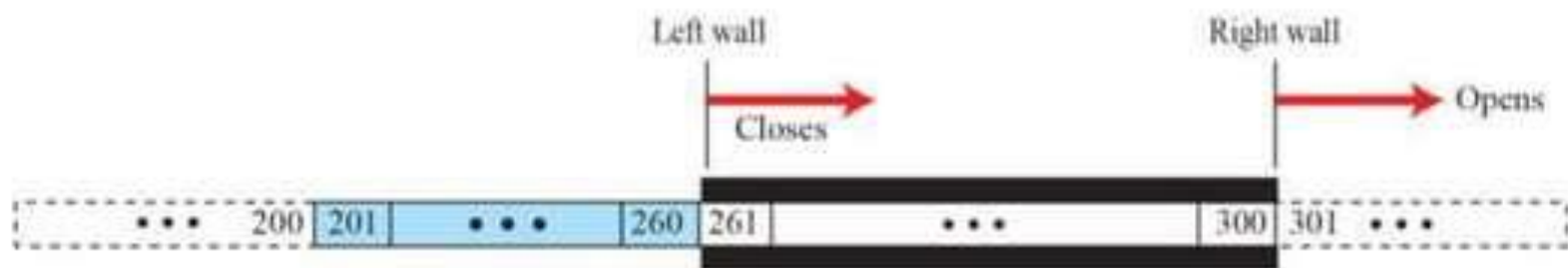


Figure 24.18: Receive window in TCP



a. Receive window and allocated buffer



b. Opening and closing of receive window



Example

What is the value of the receiver window (rwnd) for host A if the receiver, host B, has a buffer size of 5000 bytes and 1000 bytes of received and unprocessed data?

Solution

The value of $rwnd = 5000 - 1000 = 4000$. Host B can receive only 4000 bytes of data before overflowing its buffer. Host B advertises this value in its next segment to A.



24.3.7 Flow Control

As discussed before, flow control balances the rate a producer creates data with the rate a consumer can use the data. TCP separates flow control from error control. In this section we discuss flow control, ignoring error control. We assume that the logical channel between the sending and receiving TCP is error-free.

Figure 24.19: Data flow and flow control feedbacks in TCP

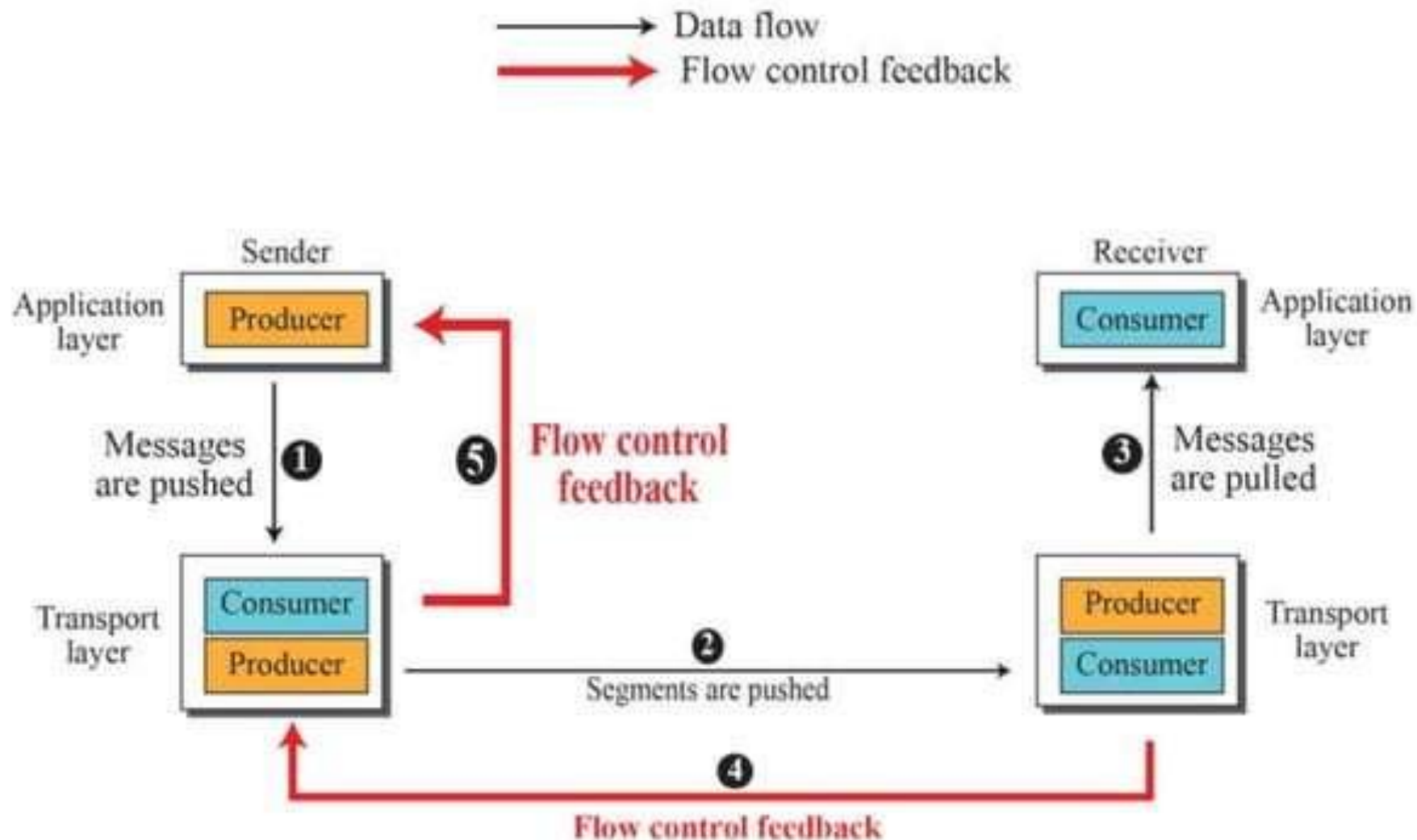
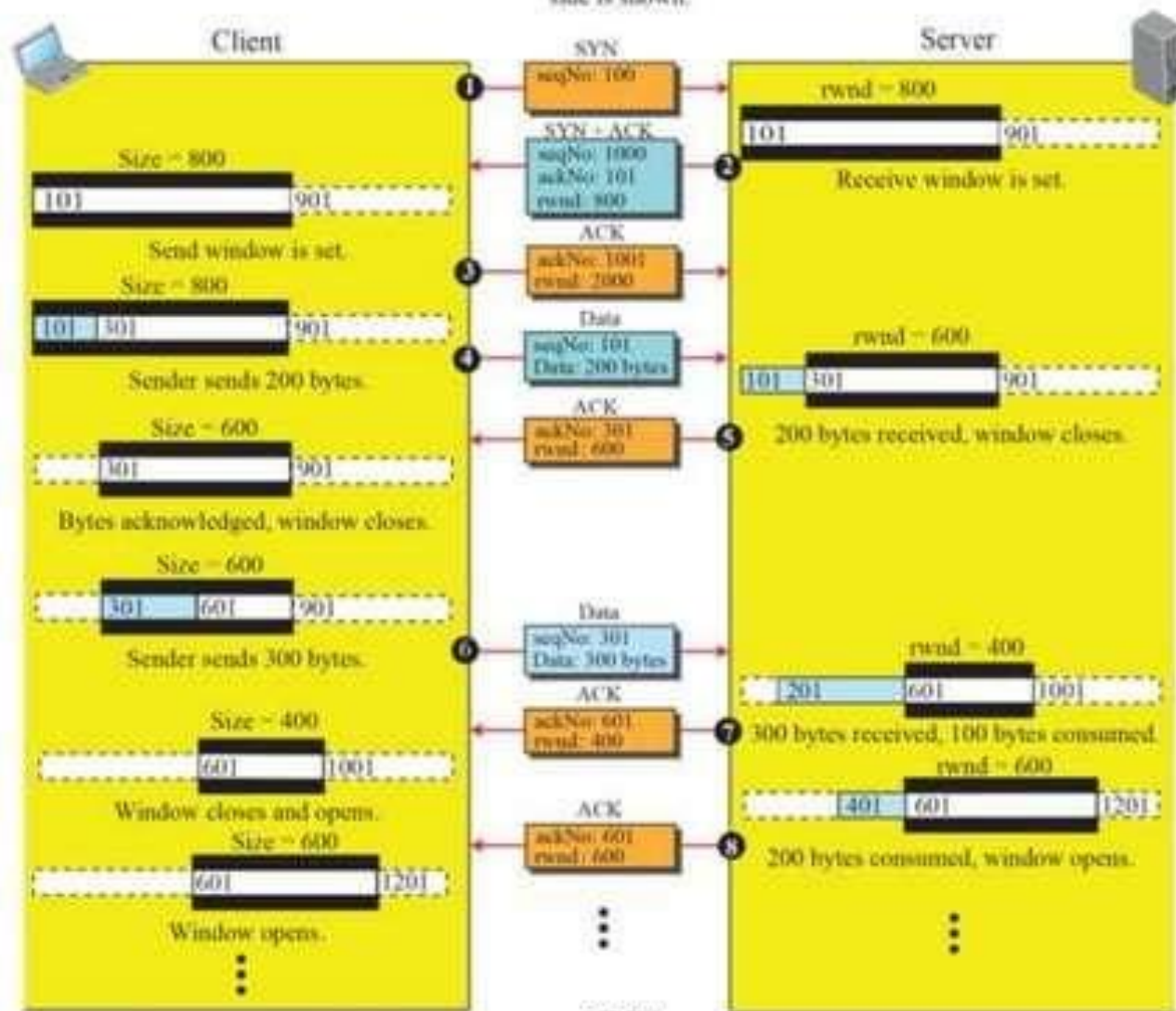


Figure 24.20: An example of flow control

Note: We assume only unidirectional communication from client to server. Therefore, only one window at each side is shown.





24.3.8 Error Control

TCP is a reliable transport-layer protocol. This means that an application program that delivers a stream of data to TCP relies on TCP to deliver the entire stream to the application program on the other end in order, without error, and without any part lost or duplicated.

Figure 24.24: Normal operation

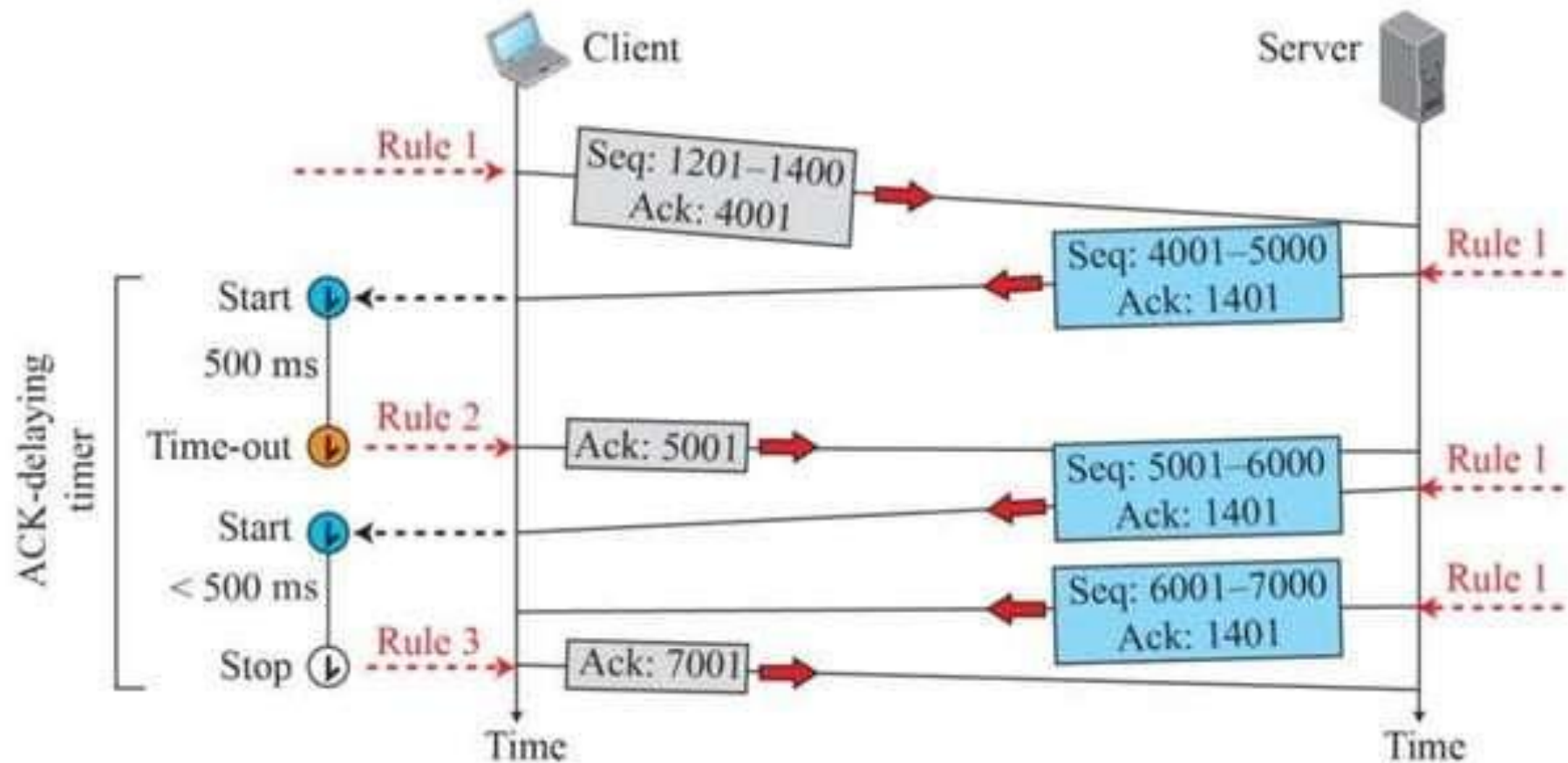


Figure 24.25: Lost segment

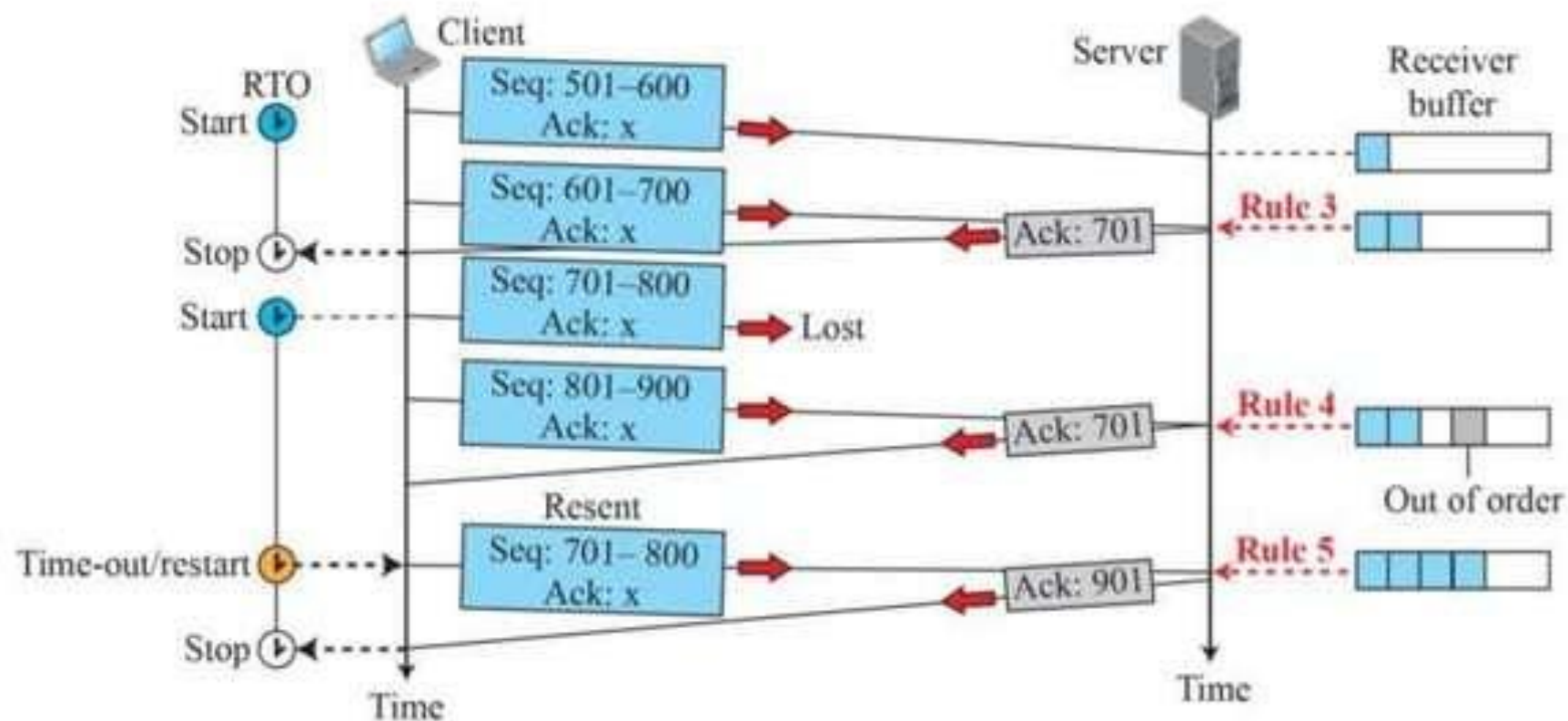


Figure 24.26: Fast retransmission

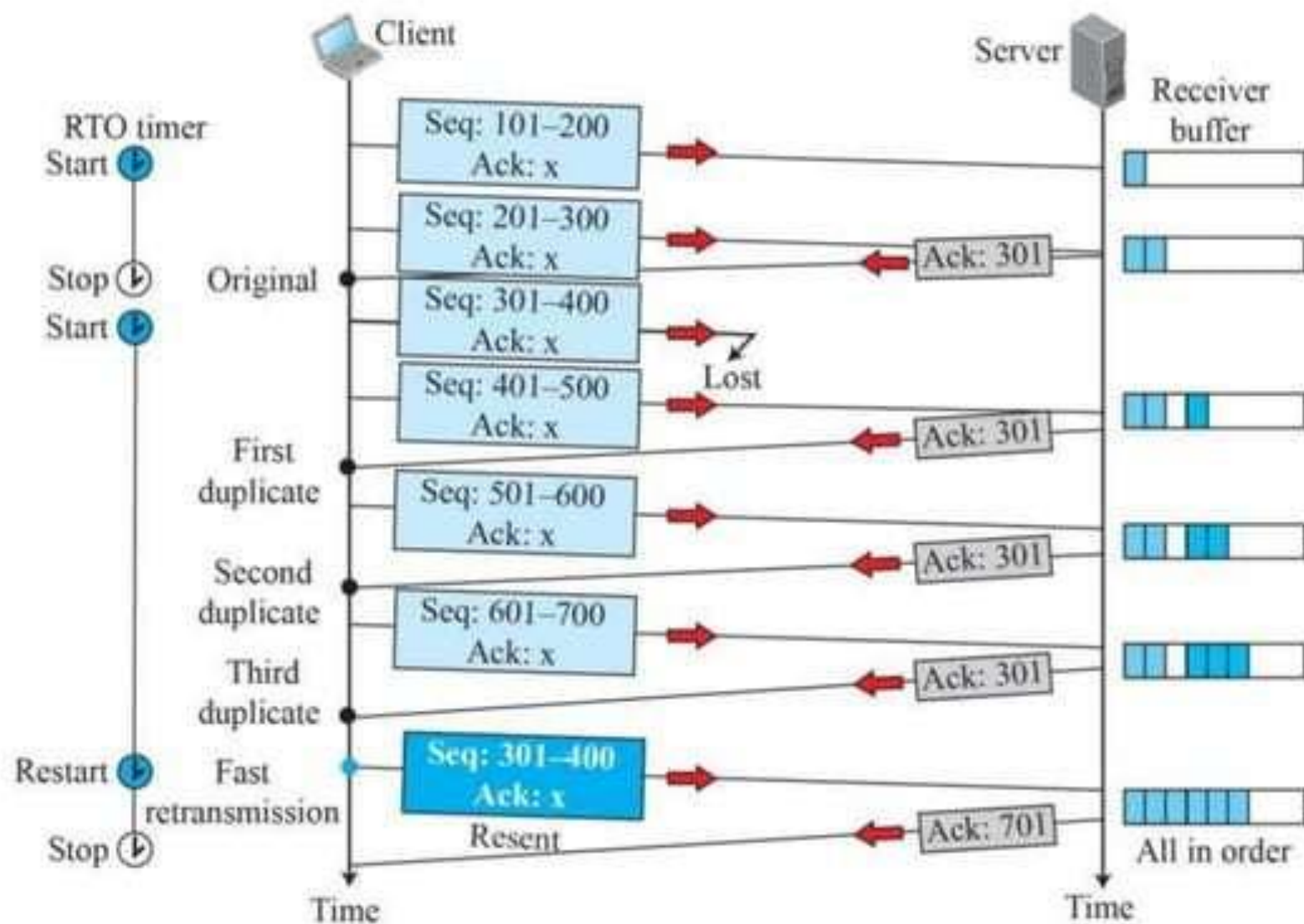


Figure 24.27: Lost acknowledgment

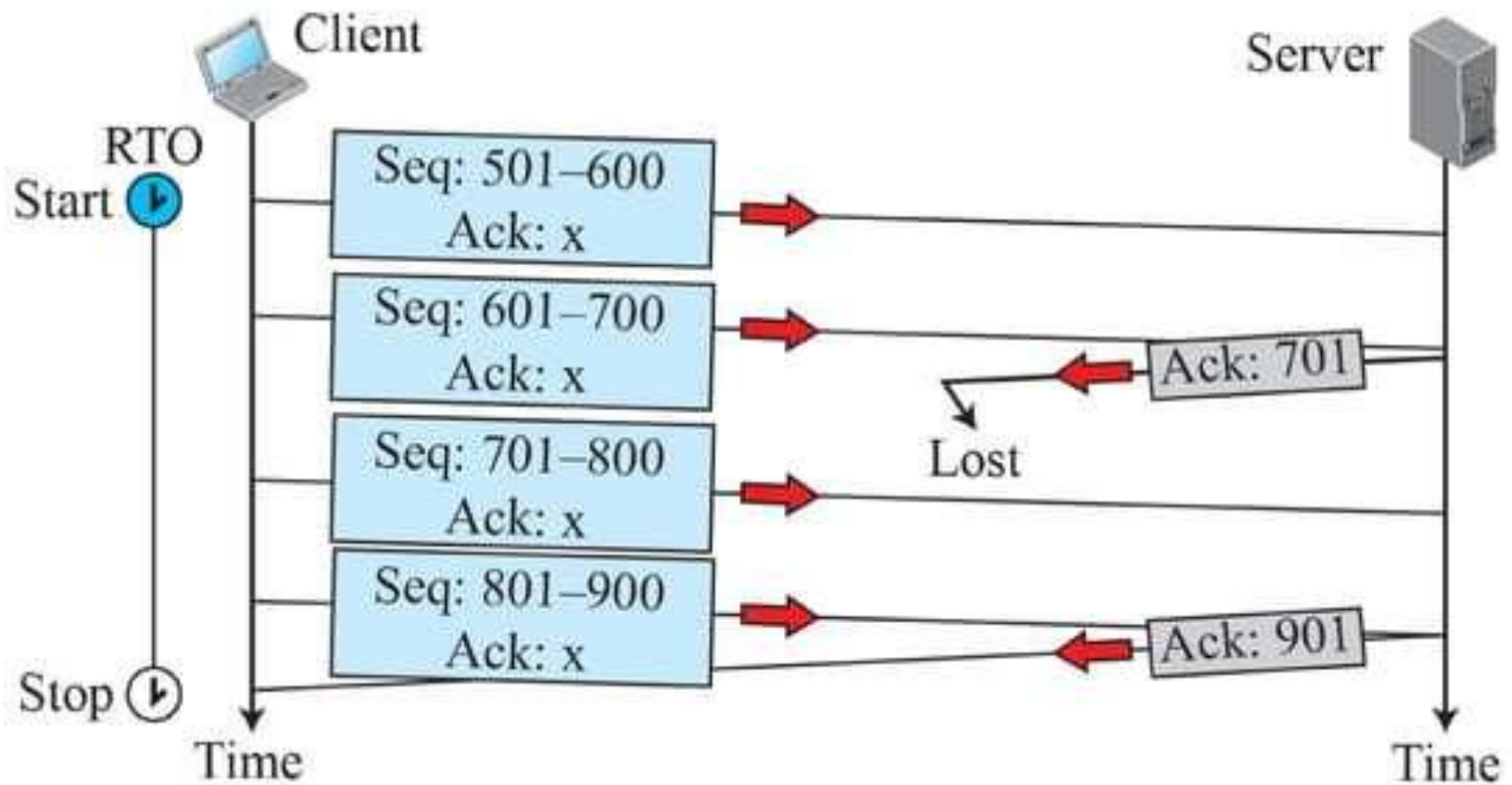
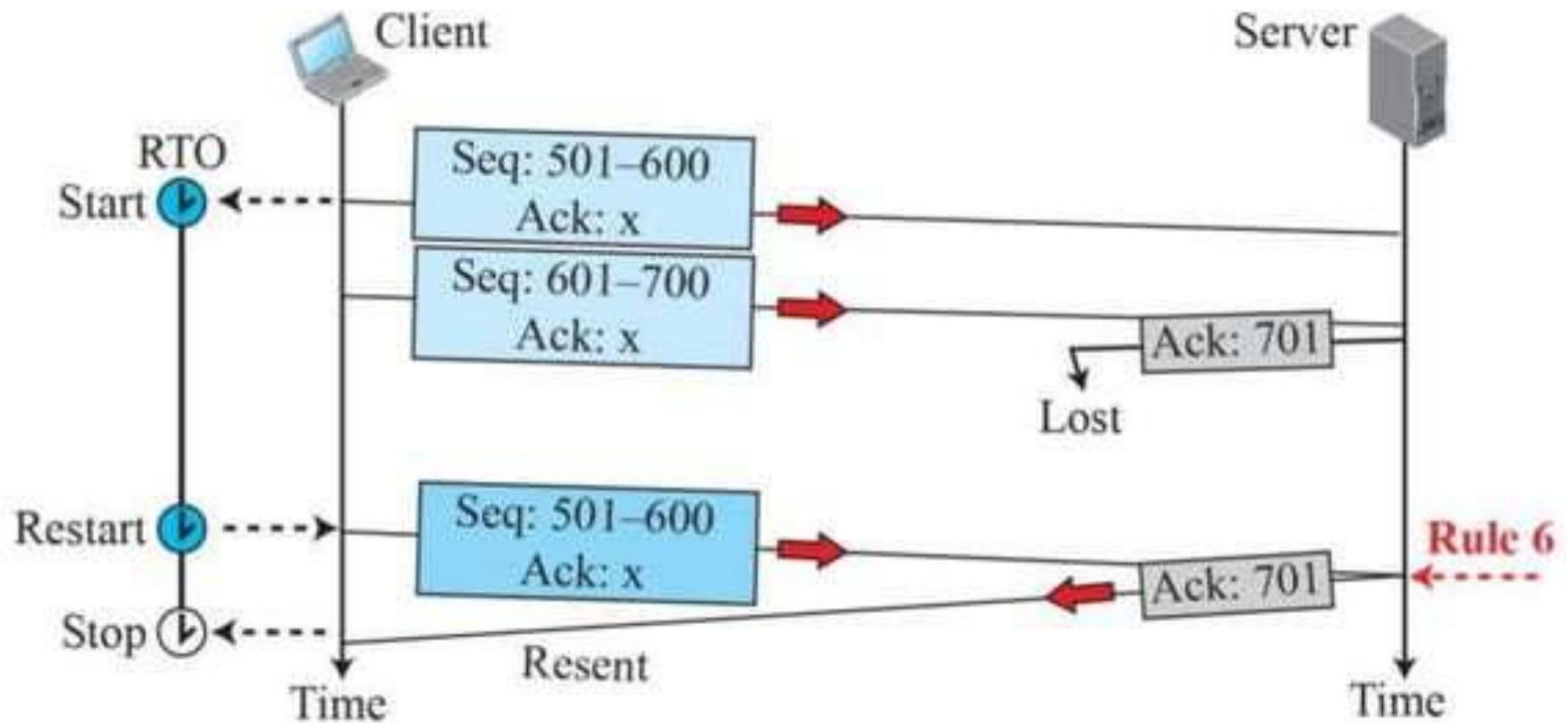


Figure 24.28: Lost acknowledgment corrected by resending a segment





24.3.9 TCP Congestion Control

TCP uses different policies to handle the congestion in the network. We describe these policies in this section.

Figure 24.29: Slow start, exponential increase

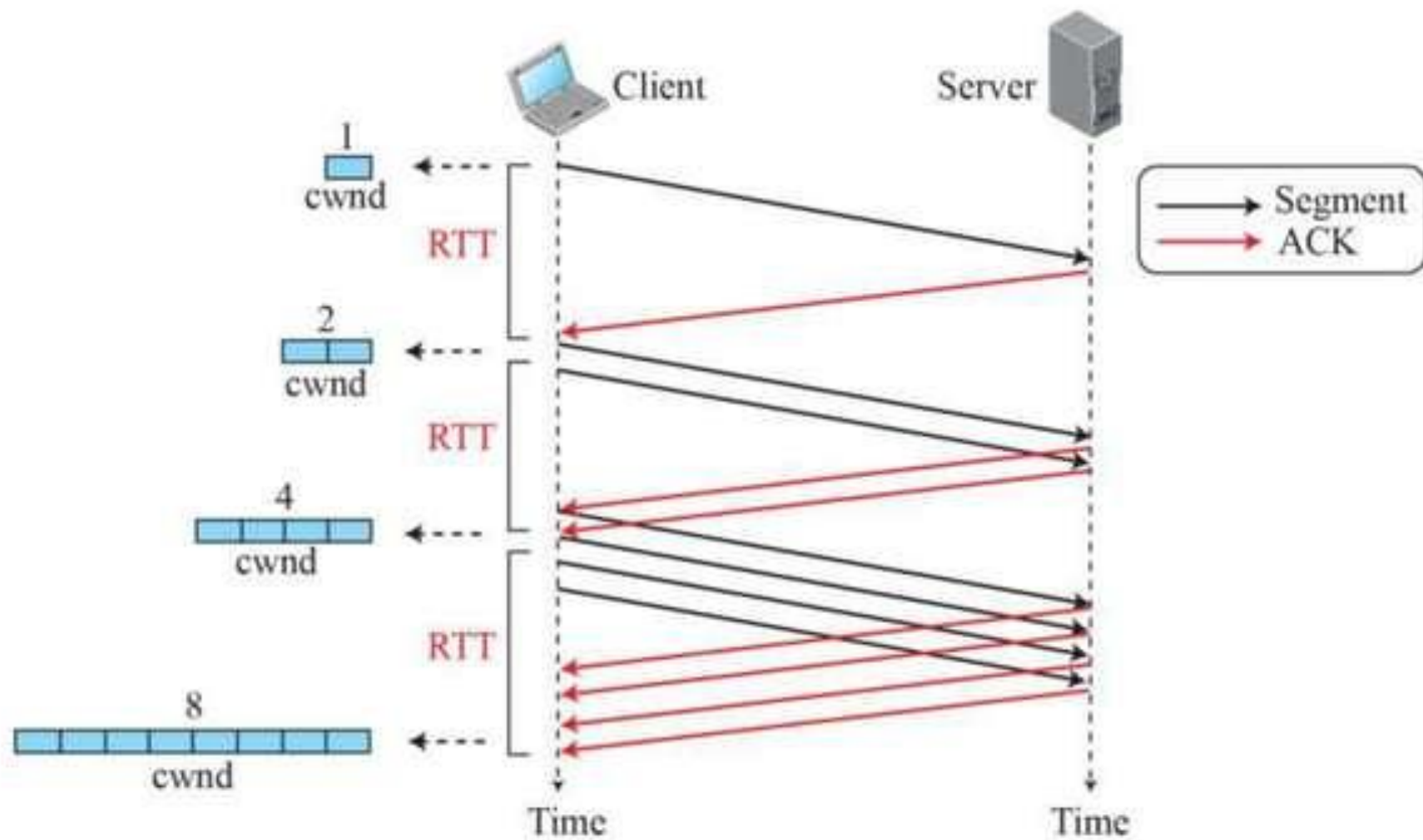


Figure 24.30: Congestion avoidance, additive increase

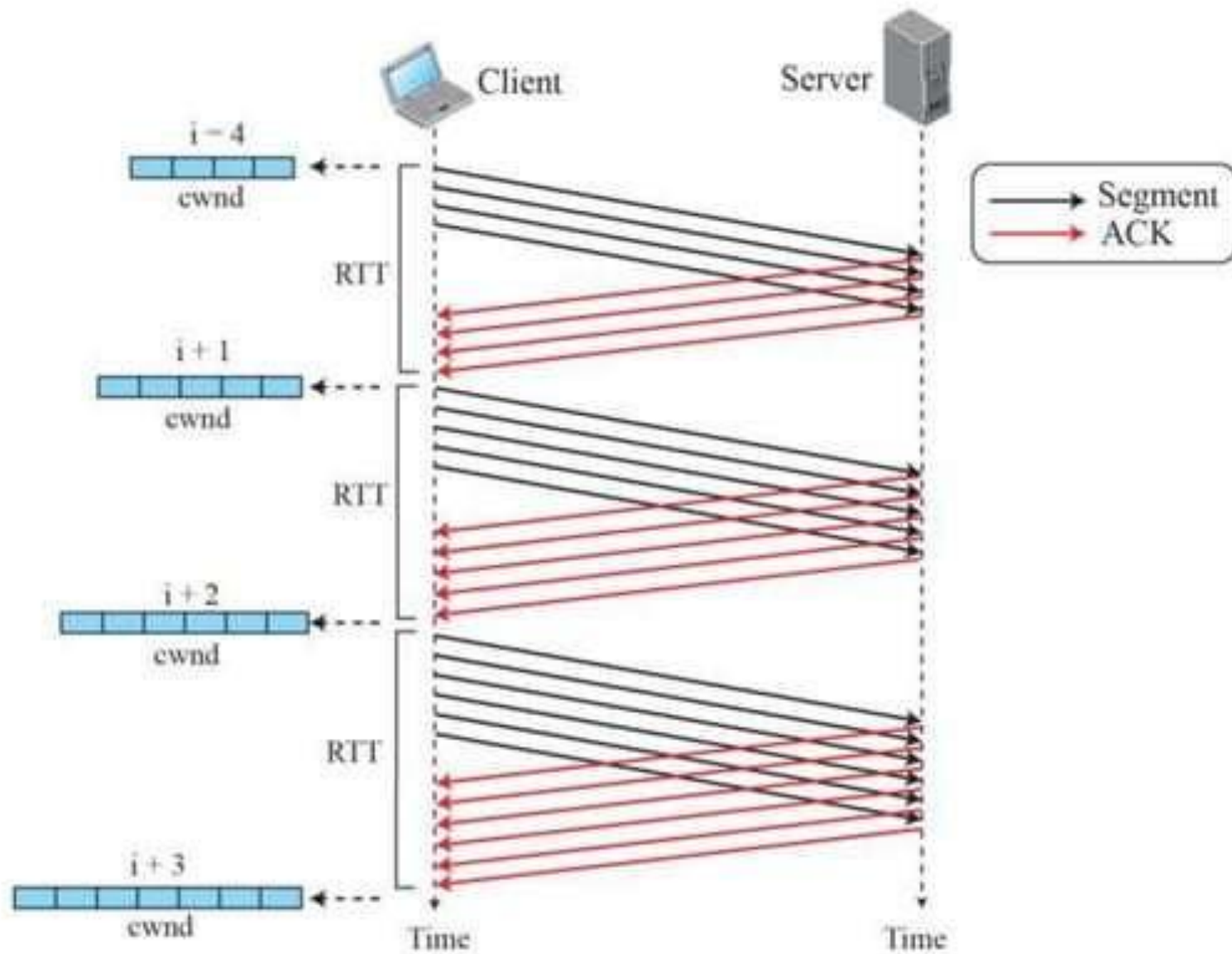


Figure 24.31: FSM for Tahoe TCP

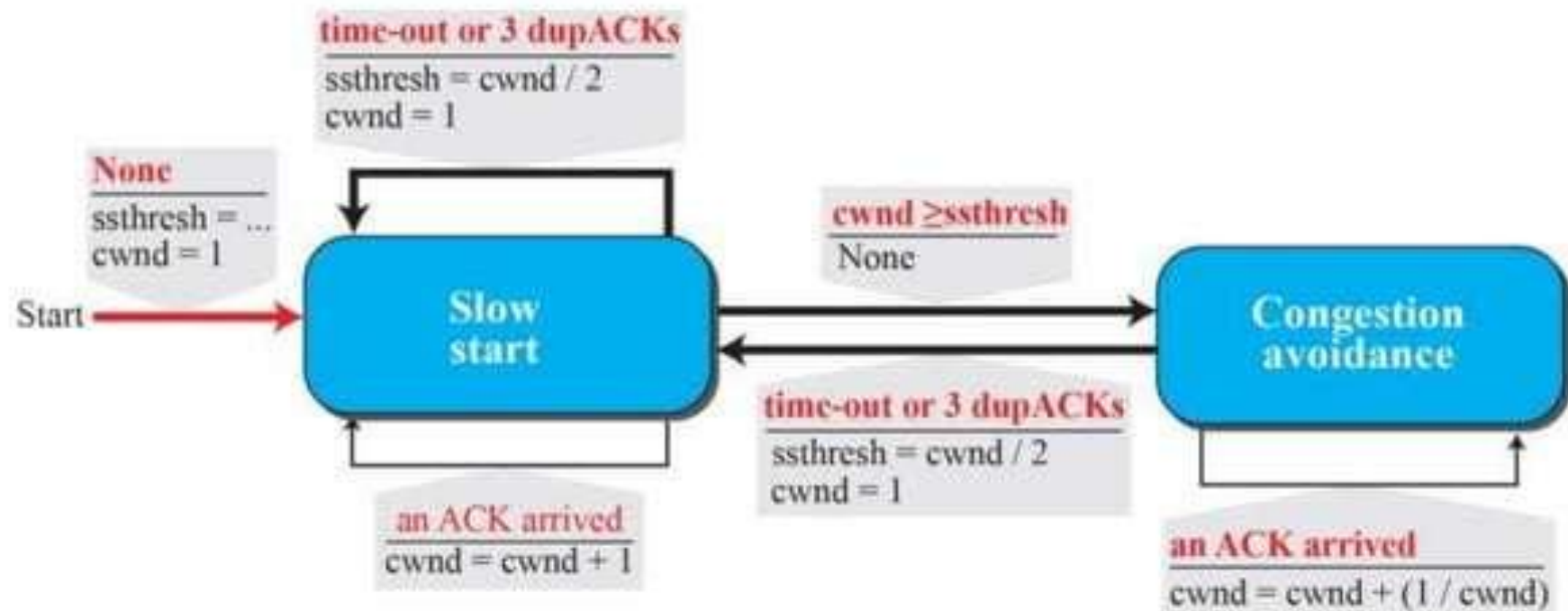
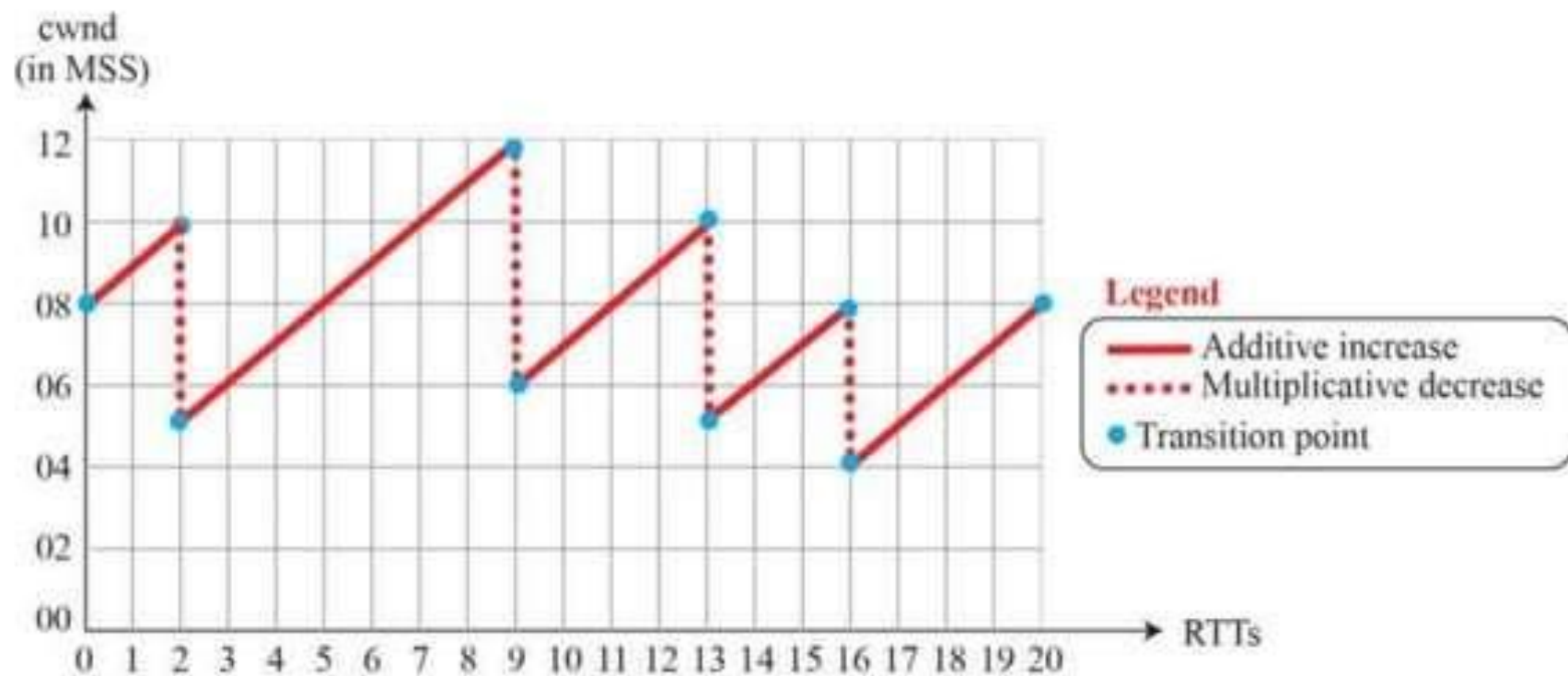


Figure 24.35: Additive increase, multiplicative decrease (AIMD)



Retransmission Timer Management

- static timer likely too long or too short
- estimate round trip delay by observing pattern of delay for recent segments
- set time to value a bit greater than estimate
- simple average over a number of segments
- exponential average using time series (RFC793)
- RTT Variance Estimation (Jacobson's algorithm)

Retransmission Timer

■ Simple Average

- **RTT(*i*):** round-trip time observed for the *i*th transmitted segment
- **ARTT(*K*):** average round-trip time for the first *K* segments

$$ARTT(K+1) = \frac{1}{K+1} \sum_{i=1}^{K+1} RTT(i)$$

$$ARTT(K+1) = \frac{K}{K+1} ARTT(K) + \frac{1}{K+1} RTT(K+1)$$

- **Exponential Average**
 - **SRTT: smoothed round-trip time estimate**
 - **RTO: retransmission timer**

$$SRTT(K+1) = \alpha \times SRTT(K) + (1 - \alpha) \times RTT(K+1)$$

$$RTO(K+1) = SRTT(K+1) + \Delta$$

RTT Variance Estimation

- **AERR(K): sample mean deviation measured at time K**

$$AERR(K+1) = RTT(K+1) - ARTT(K)$$

$$\begin{aligned} ADEV(K+1) &= \frac{1}{K+1} \sum_{i=1}^{K+1} |AERR(i)| \\ &= \frac{K}{K+1} ADEV(K) + \frac{1}{K+1} |AERR(K+1)| \end{aligned}$$

RTT Variance Estimation

■ Jacobson's Algorithm

$$SRTT(K+1) = (1-g) \times SRTT(K) + g \times RTT(K+1)$$

$$SERR(K+1) = RTT(K+1) - SRTT(K)$$

$$SDEV(K+1) = (1-h) \times SDEV(K) + h \times |SERR(K+1)|$$

$$RTO(K+1) = SRTT(K+1) + f \times SDEV(K+1)$$

$$\mathbf{g = 1/8 = 0.125, h = 1/4 = 0.25, f = 2}$$

Jacobson's RTO

■ Smoothed RTT

Initially	→	No value
After first measurement	→	$RTT_S = RTT_M$
After each measurement	→	$RTT_S = (1 - \alpha) RTT_S + \alpha \times RTT_M$

■ RTT Deviation

Initially	→	No value
After first measurement	→	$RTT_D = RTT_M / 2$
After each measurement	→	$RTT_D = (1 - \beta) RTT_D + \beta \times RTT_S - RTT_M $

■ RTO

Original	→	Initial value
After any measurement	→	$RTO = RTT_S + 4 \times RTT_D$

Exponential RTO Backoff

- timeout probably due to congestion
 - dropped packet or long round trip time
- hence maintaining RTO is not good idea
- better to increase RTO each time a segment is re-transmitted
 - $RTO = q * RTO$
 - commonly $q=2$ (binary exponential backoff)
 - as in ethernet CSMA/CD

Karn's Algorithm

- if segment is re-transmitted, ACK may be for:
 - first copy of the segment (longer RTT than expected)
 - second copy
- no way to tell
- don't measure RTT for re-transmitted segments
- calculate backoff when re-transmission occurs
- use backoff RTO until ACK arrives for segment that has not been re-transmitted

24-4 SCTP

Stream Control Transmission Protocol (SCTP) is a new transport-layer protocol designed to combine some features of UDP and TCP in an effort to create a protocol for multimedia communication.



24.4.1 SCTP Services

Before discussing the operation of SCTP, let us explain the services offered by SCTP to the application-layer processes.

Figure 24.38 : Multiple-stream concept

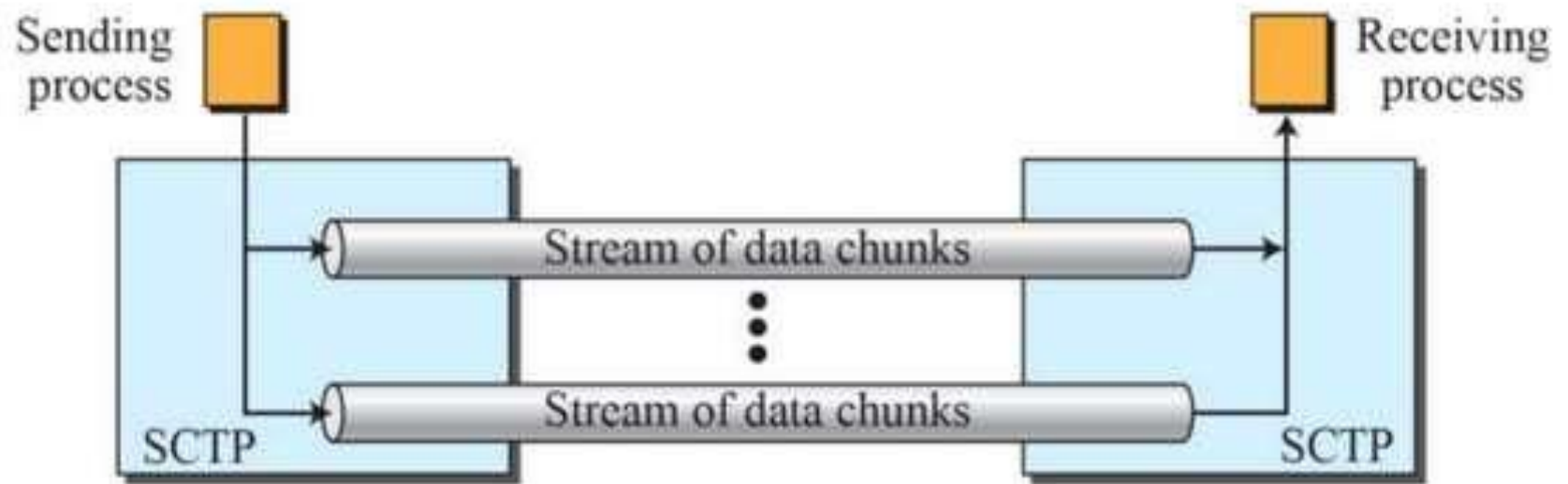
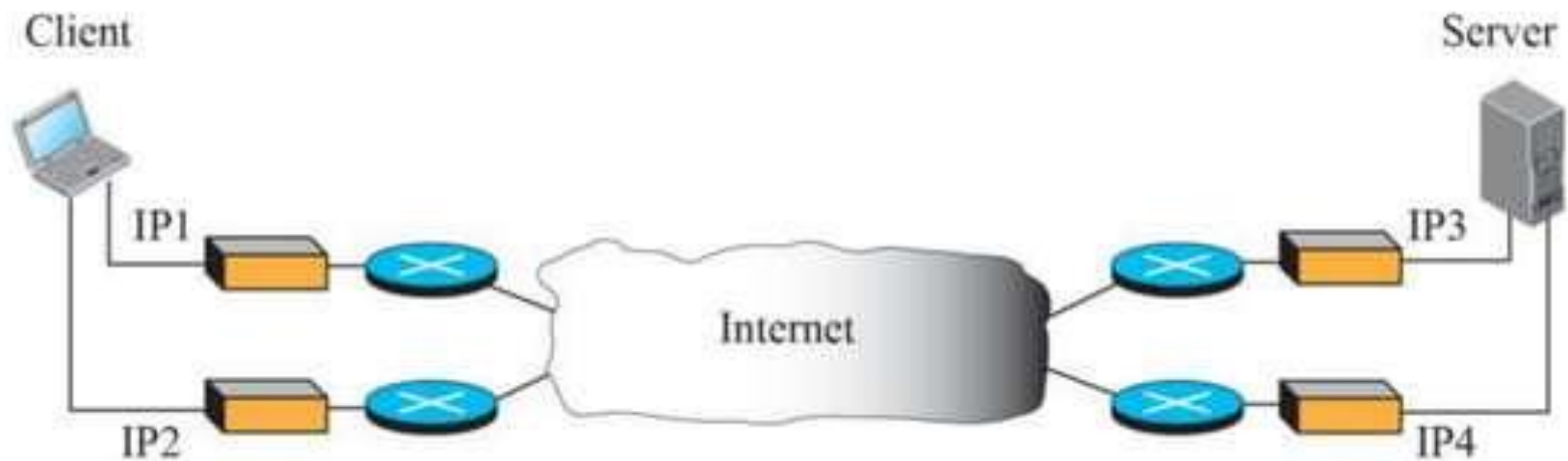


Figure 24.39 : Multihoming concept





24.4.2 *SCTP Features*

The following shows the general features of SCTP.

***Transmission Sequence
Number (TSN)
Stream Identifier (SI)***

***Stream Sequence Number
(SSN)***

Figure 24.40 : Comparison between a TCP segment and an SCTP packet

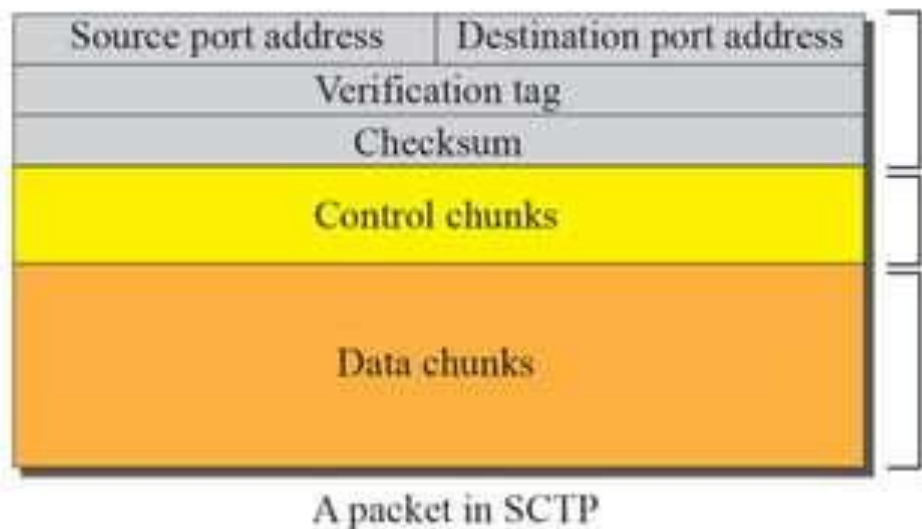
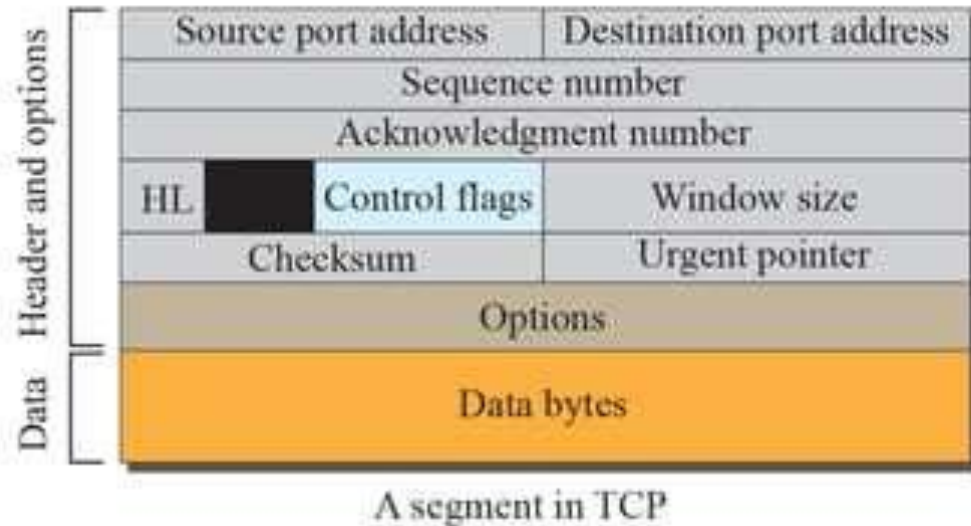
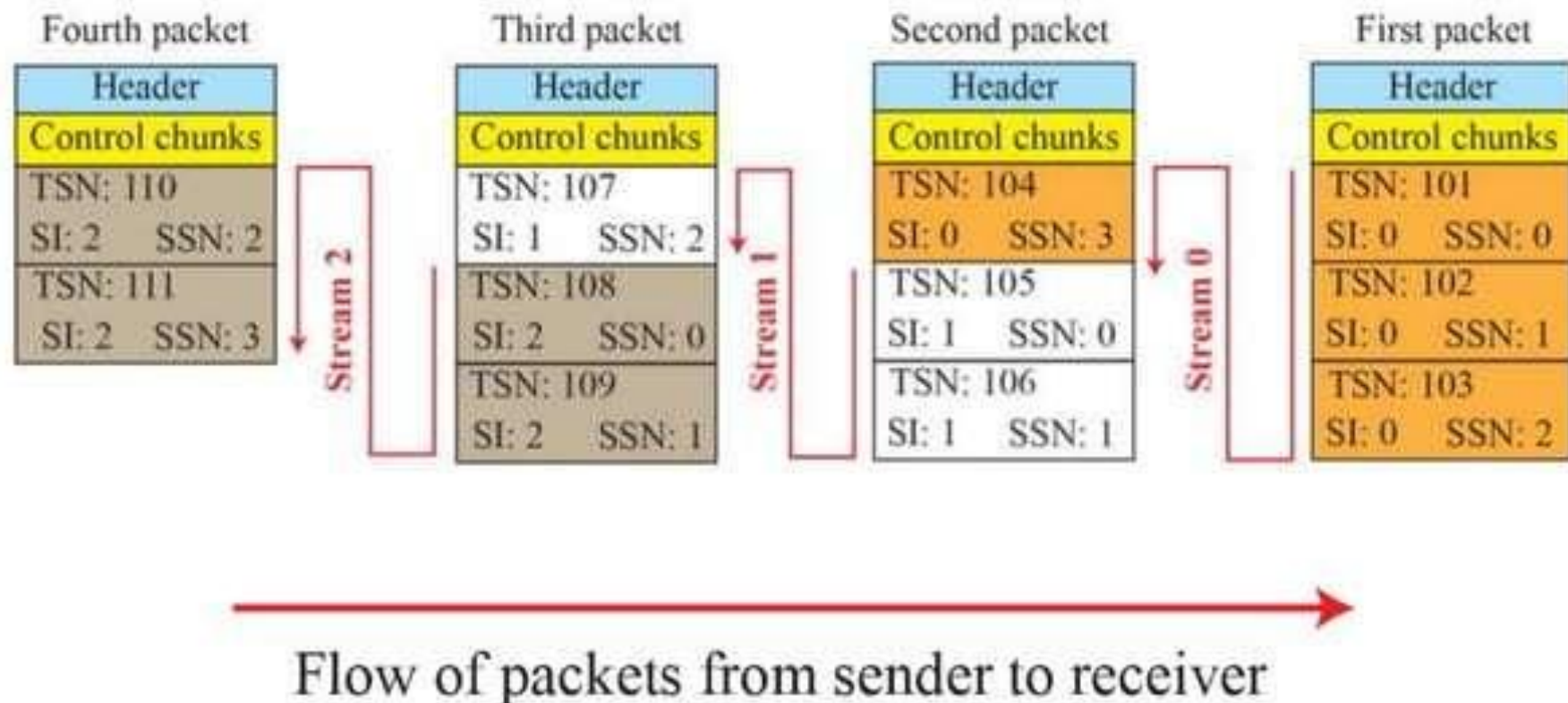


Figure 24.41 : Packets, data chunks, and streams





24.4.3 Packet Format

An SCTP packet has a mandatory general header and a set of blocks called chunks. There are two types of chunks: control chunks and data chunks. A control chunk controls and maintains the association; a data chunk carries user data. In a packet, the control chunks come before the data chunks. Figure 24.42 shows the general format of an SCTP packet.

Figure 24.43 : SCTP packet format

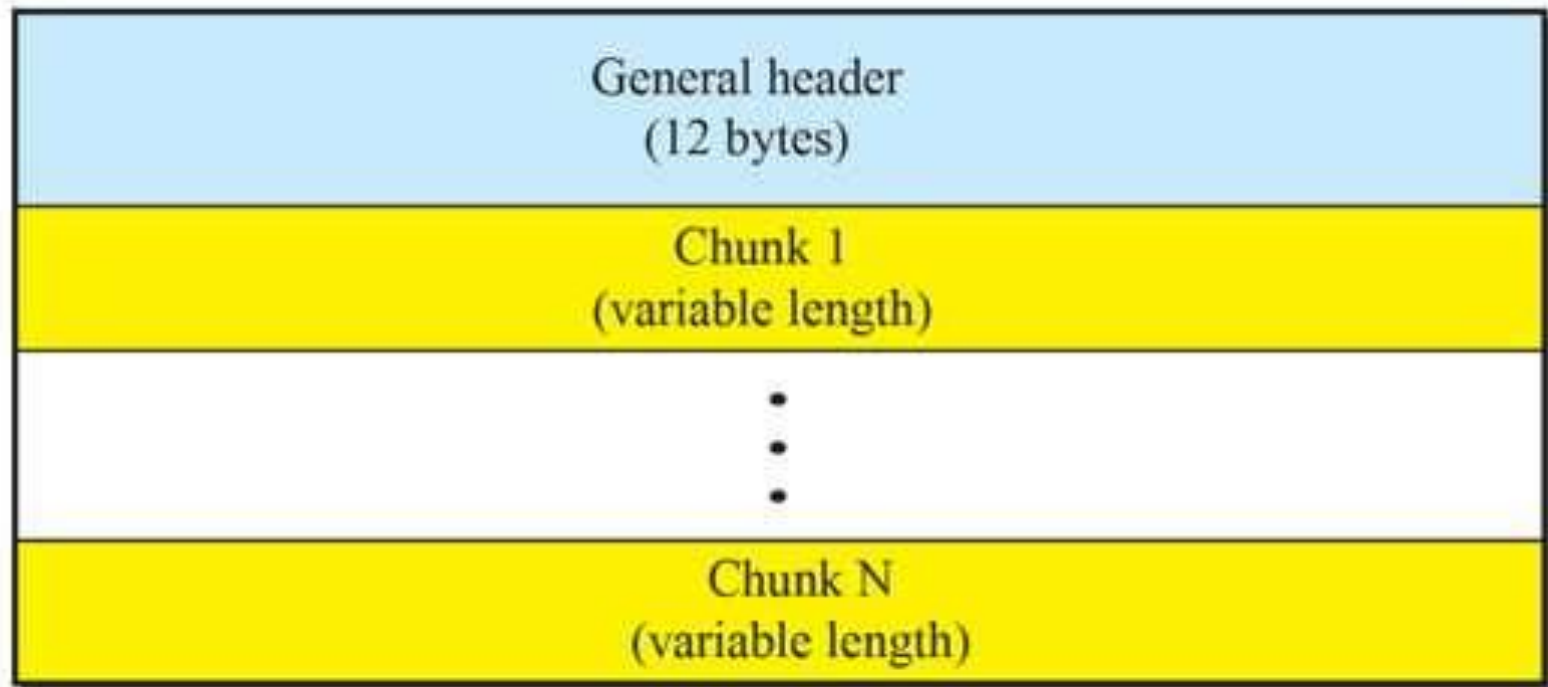
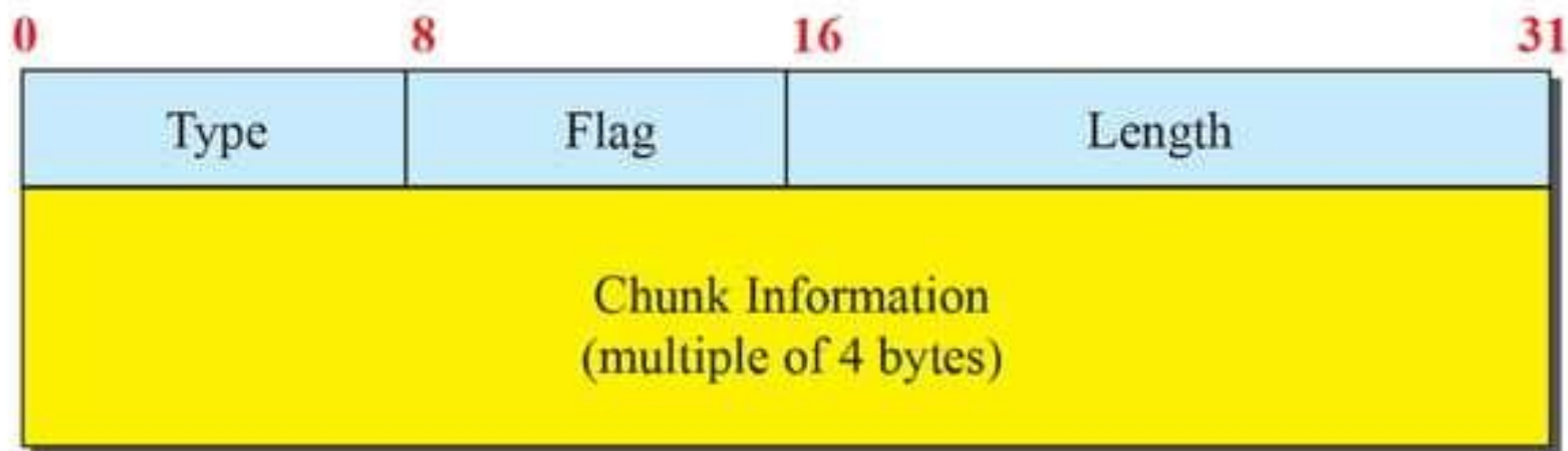
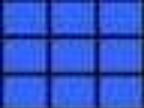


Figure 8.50 : General header

Source port address 16 bits	Destination port address 16 bits
Verification tag 32 bits	
Checksum 32 bits	

Figure 24.44 : Common layout of a chunk



 **Table 24.3: Chunks**

<i>Type</i>	<i>Chunk</i>	<i>Description</i>
0	DATA	User data
1	INIT	Sets up an association
2	INIT ACK	Acknowledges INIT chunk
3	SACK	Selective acknowledgment
4	HEARTBEAT	Probes the peer for liveliness
5	HEARTBEAT ACK	Acknowledges HEARTBEAT chunk
6	ABORT	Aborts an association
7	SHUTDOWN	Terminates an association
8	SHUTDOWN ACK	Acknowledges SHUTDOWN chunk
9	ERROR	Reports errors without shutting down
10	COOKIE ECHO	Third packet in association establishment
11	COOKIE ACK	Acknowledges COOKIE ECHO chunk
14	SHUTDOWN COMPLETE	Third packet in association termination
192	FORWARD TSN	For adjusting cumulating TSN



24.4.4 *An SCTP Association*

SCTP, like TCP, is a connection-oriented protocol. However, a connection in SCTP is called an association to emphasize multihoming.

Figure 24.45: Four-way handshaking

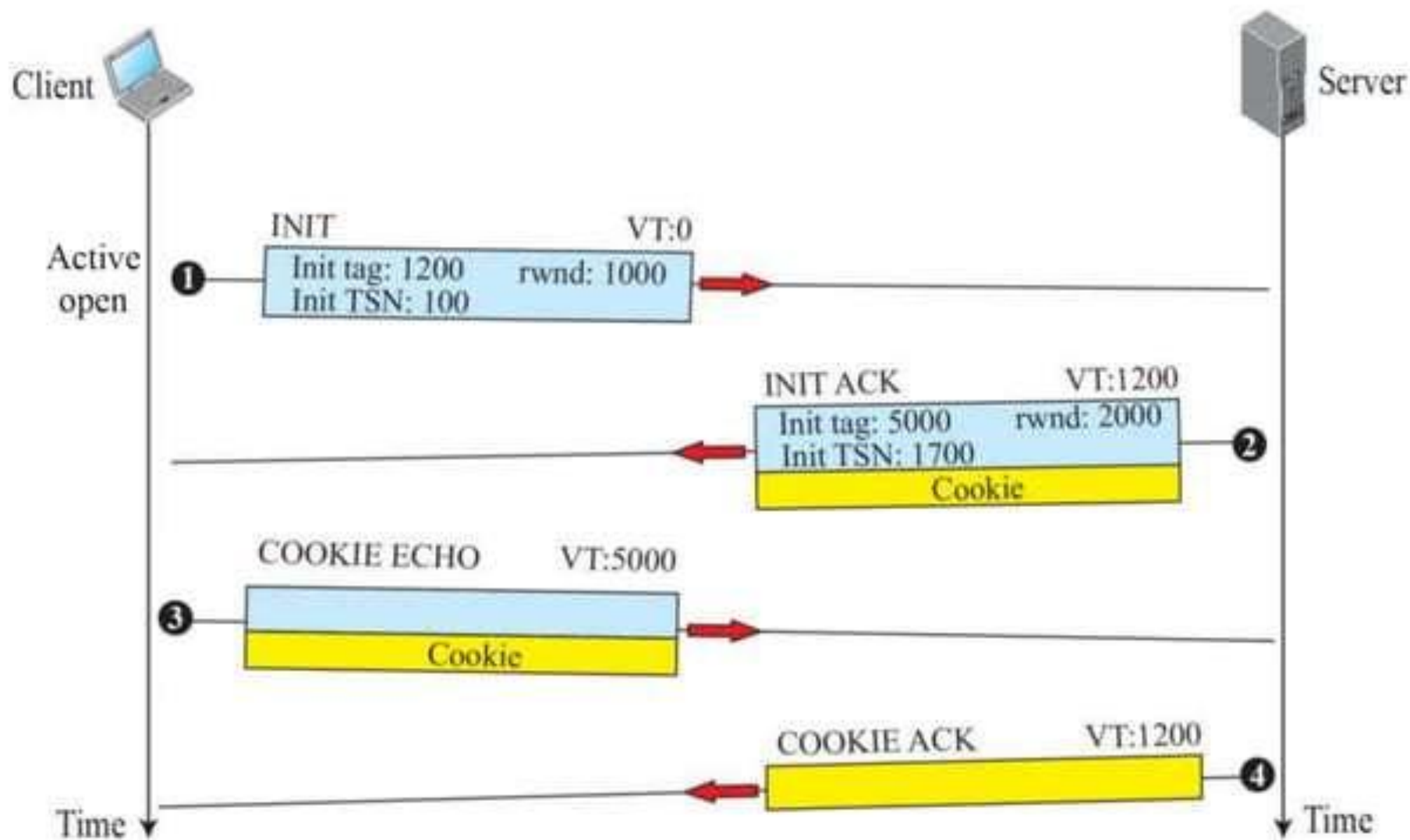
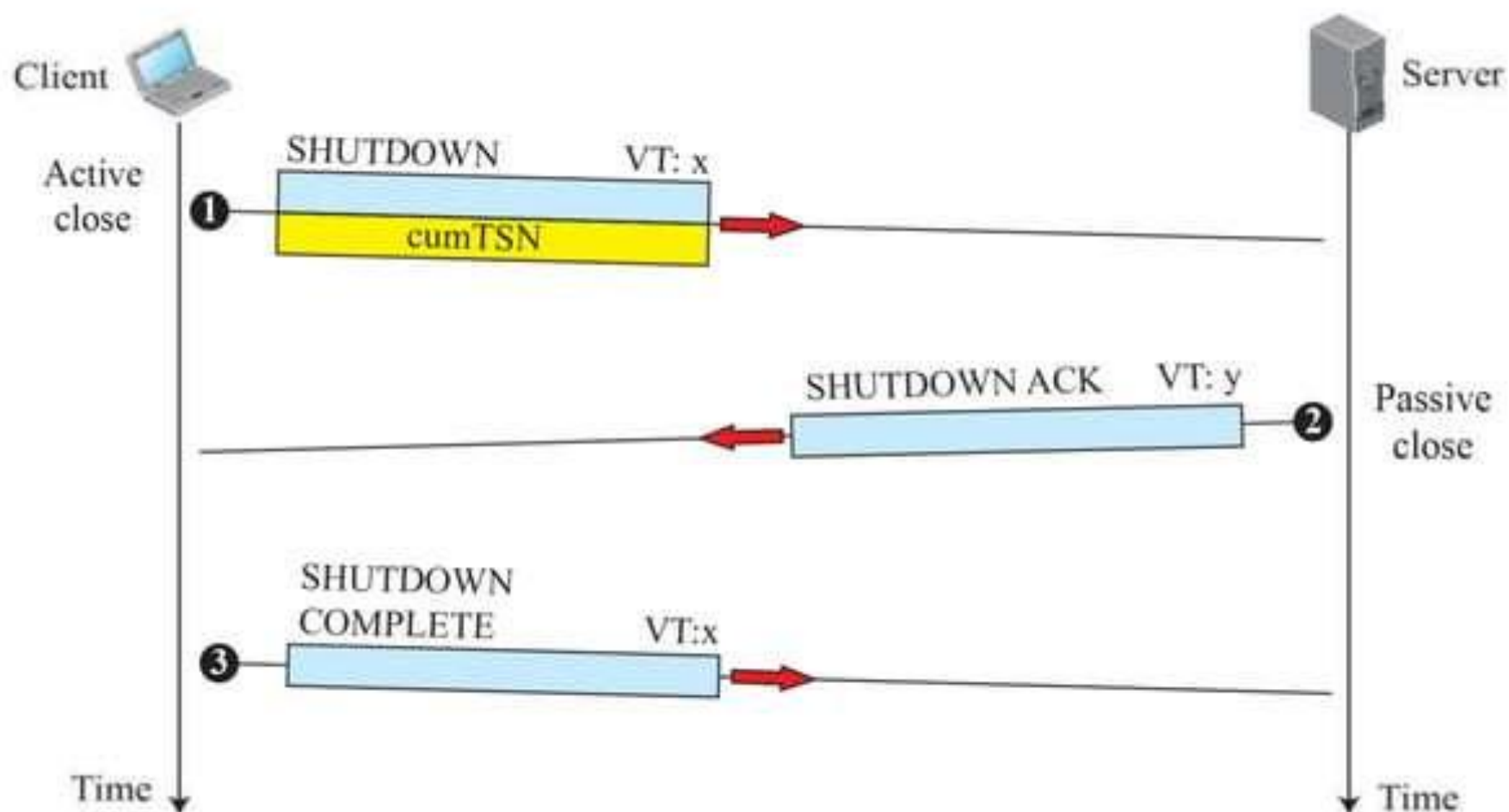


Figure 24.46 : Association termination





24.4.5 Flow Control

Flow control in SCTP is similar to that in TCP. In SCTP, we need to handle two units of data, the byte and the chunk. The values of rwnd and cwnd are expressed in bytes; the values of TSN and acknowledgments are expressed in chunks. To show the concept, we make some unrealistic assumptions. We assume that there is never congestion in the network and that the network is error free.

Figure 24.47: Flow control, receiver site

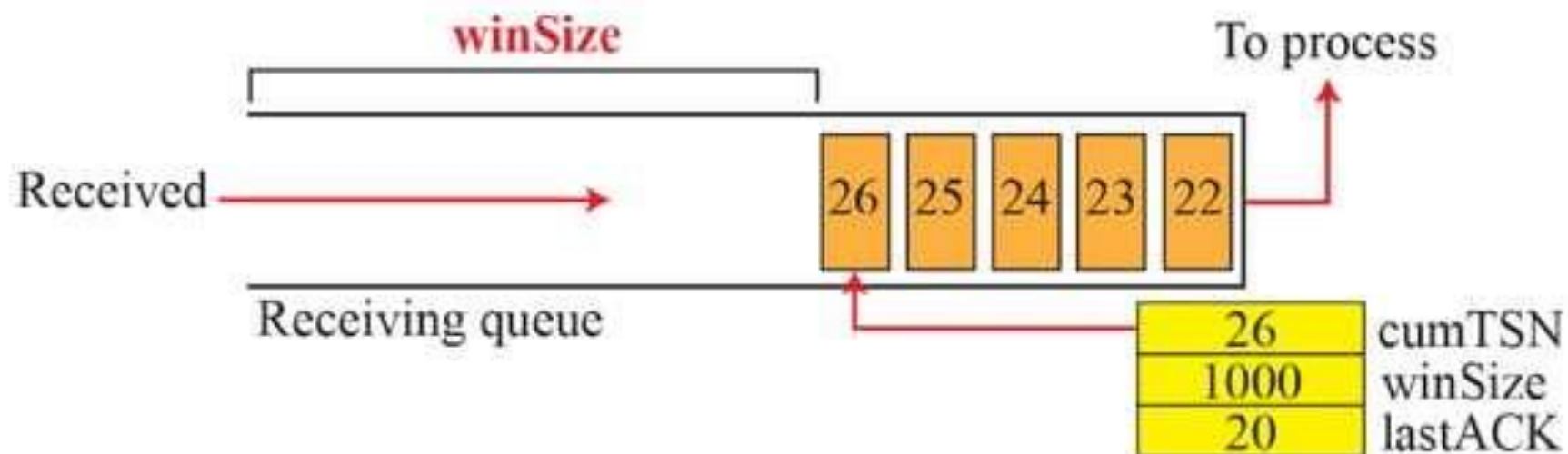
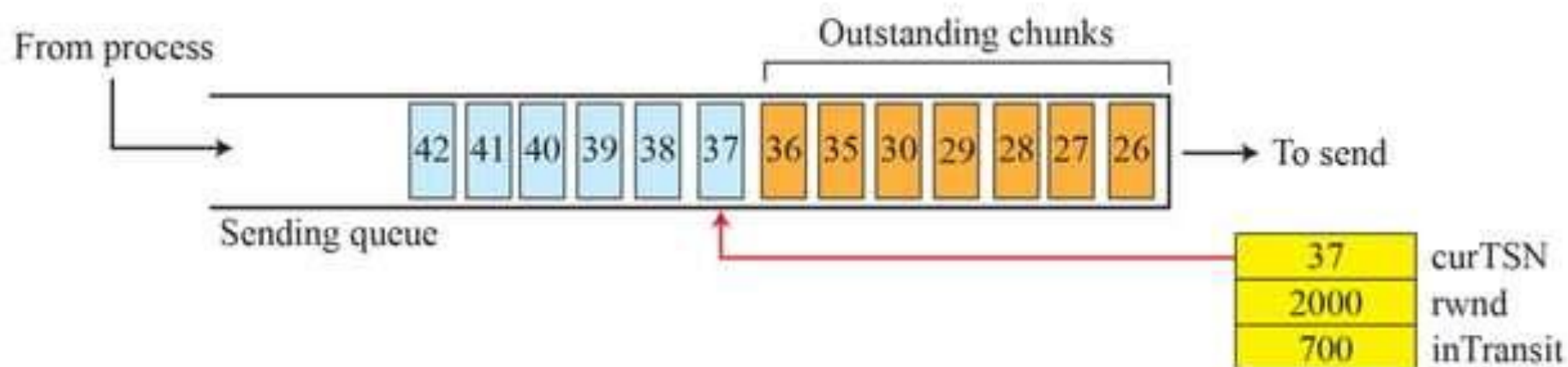


Figure 24.48: Flow control, sender site

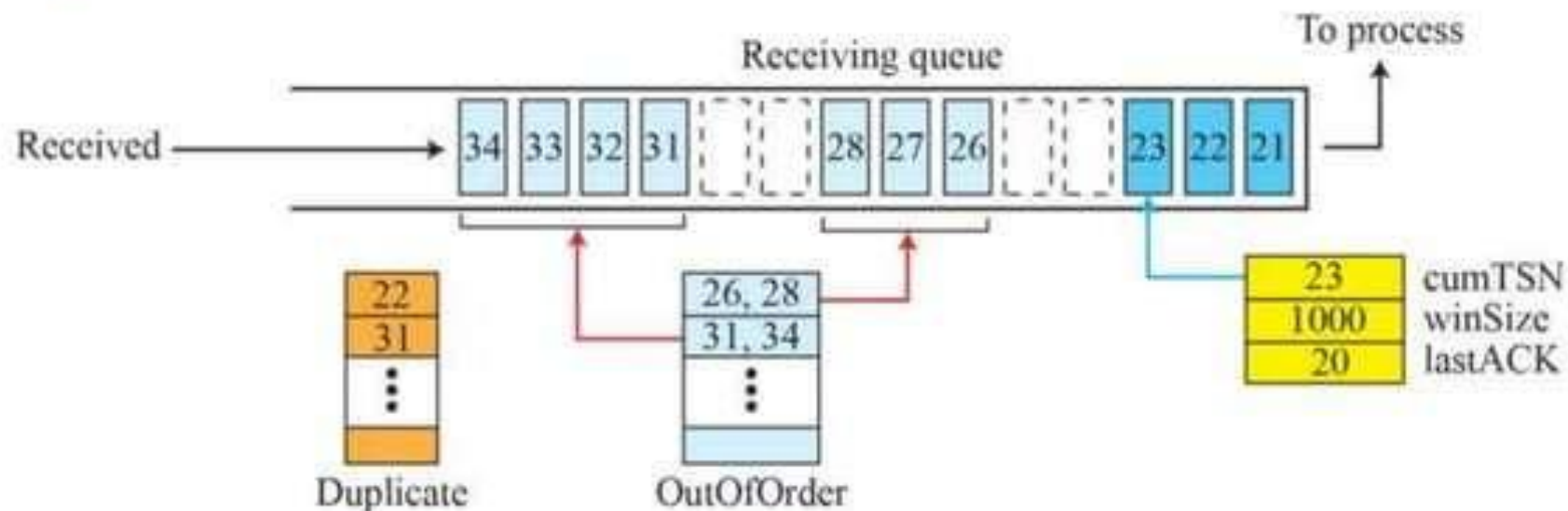




24.4.6 Error Control

SCTP, like TCP, is a reliable transport-layer protocol. It uses a SACK chunk to report the state of the receiver buffer to the sender. Each implementation uses a different set of entities and timers for the receiver and sender sites. We use a very simple design to convey the concept to the reader.

Figure 24.49 : Error control, receiver site



SACK chunk

Type: 3	Flag: 0	Length: 32
Cumulative TSN: 23		
Advertised receiver window credit: 1000		
Number of gap ACK blocks: 2		Number of duplicates: 2
Gap ACK block #1 start: 3		Gap ACK block #1 end: 5
Gap ACK block #2 start: 8		Gap ACK block #2 end: 11
Duplicate TSN: 22		
Duplicate TSN: 31		

Numbers are relative to cumTSN

Figure 24.50 : Error control, sender site

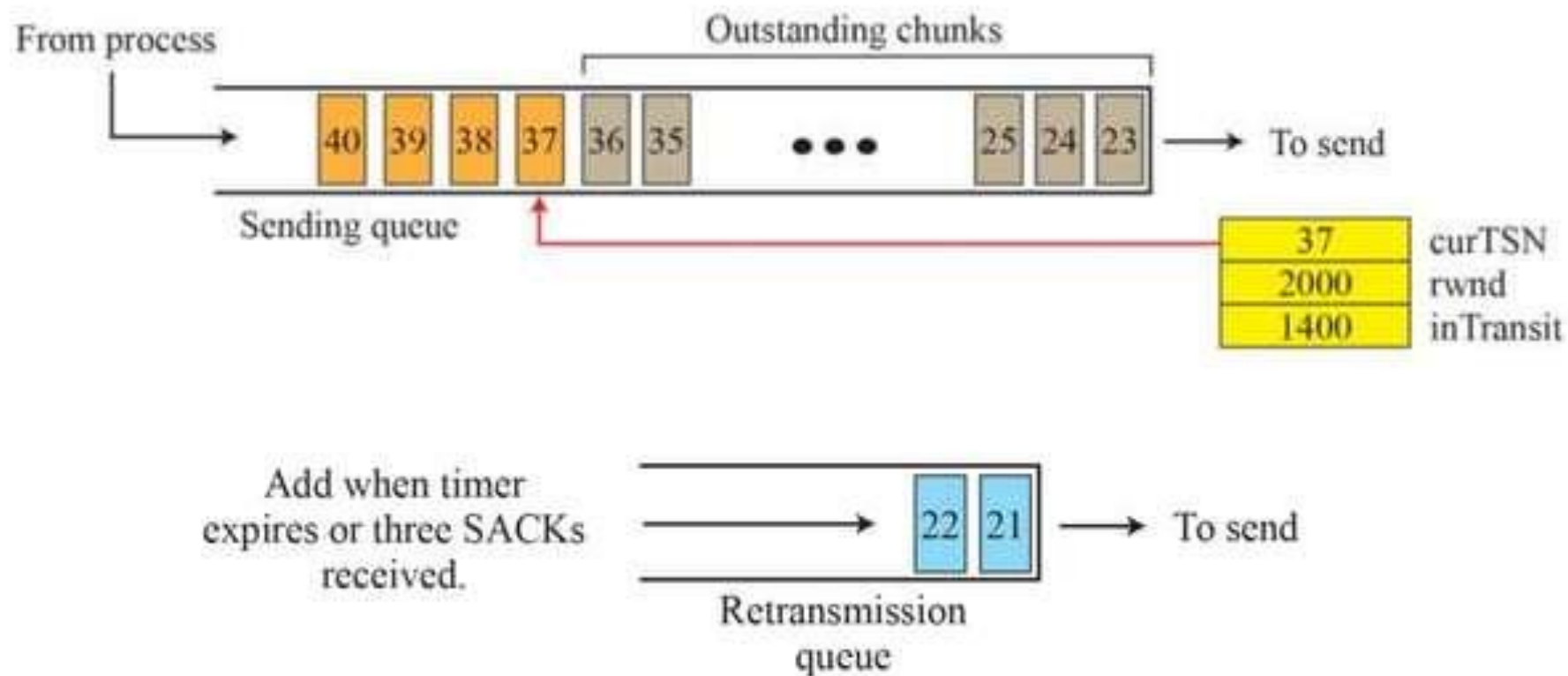


Figure 24.51: *New state at the sender site after receiving a SACK chunk*

